

Deep Learning

Lecture 2

Recap of Last Lecture

- Deep Learning – A subclass of machine learning
- Supervised Learning
 - Classification
 - Regression
 - Linear regression
 - Single variable
 - Multiple variable
 - Loss function and cost function
 - Gradient descent

Agenda

- Classification
 - Logistic Regression
 - Sigmoid function
 - Cross entropy loss function
- Neural Networks
 - Perceptron
 - Layers
 - Neural Networks as universal approximator

Logistic Regression

Classification

Classification

Email: Spam / Not Spam?

Online Transactions: Fraudulent (Yes / No)?

Tumor: Malignant / Benign ?

$$y \in \{0, 1\}$$

0: “Negative Class” (e.g., benign tumor)

1: “Positive Class” (e.g., malignant tumor)

Binary Classification



1 (cat) vs 0 (non cat)

		Blue			
Green		255	134	93	22
Red	255	134	202	22	2
	255	231	42	22	4
	123	94	83	2	192
	34	44	187	92	34
	34	76	232	124	94
	67	83	194	202	

Classification: $y = 0$ or 1

$h_{\theta}(x)$ can be > 1 or < 0

Logistic Regression: $0 \leq h_{\theta}(x) \leq 1$

Logistic Regression

Hypothesis Representation

(How the function will look like?)

Logistic Regression Model

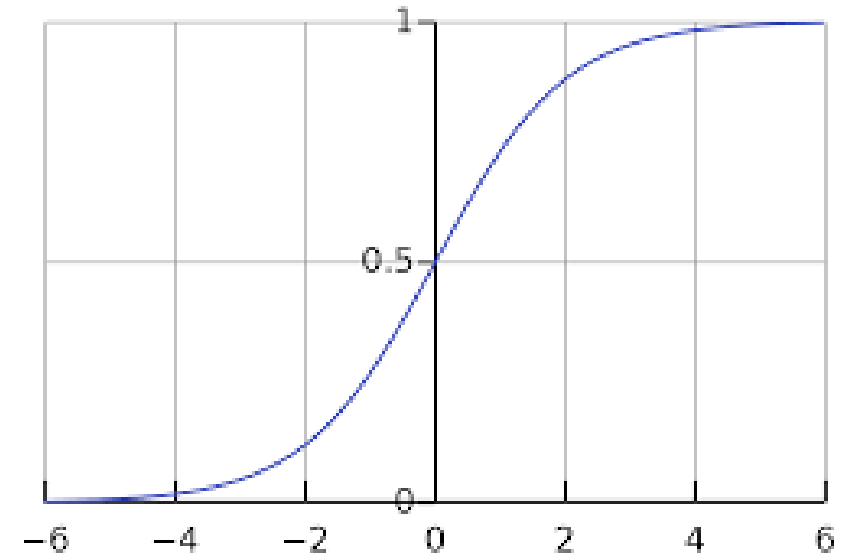
Want $0 \leq h_{\theta}(x) \leq 1$

$$h_{\theta}(x) = \theta^T x$$

$$h_{\theta}(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid function
Logistic function



Interpretation of Hypothesis Output

$h_{\theta}(x)$ = estimated probability that $y = 1$ on input x

Example: If $x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{tumorSize} \end{bmatrix}$

$$h_{\theta}(x) = 0.7$$

Tell patient that 70% chance of tumor being malignant

“probability that $y = 1$, given x ,
parameterized by θ ”

$$P(y = 0|x; \theta) + P(y = 1|x; \theta) = 1$$

$$P(y = 0|x; \theta) = 1 - P(y = 1|x; \theta)$$

Logistic Regression

Decision Boundary

(Talking about the decision boundary will give us a better sense of what the logistic regression hypothesis function is computing.)

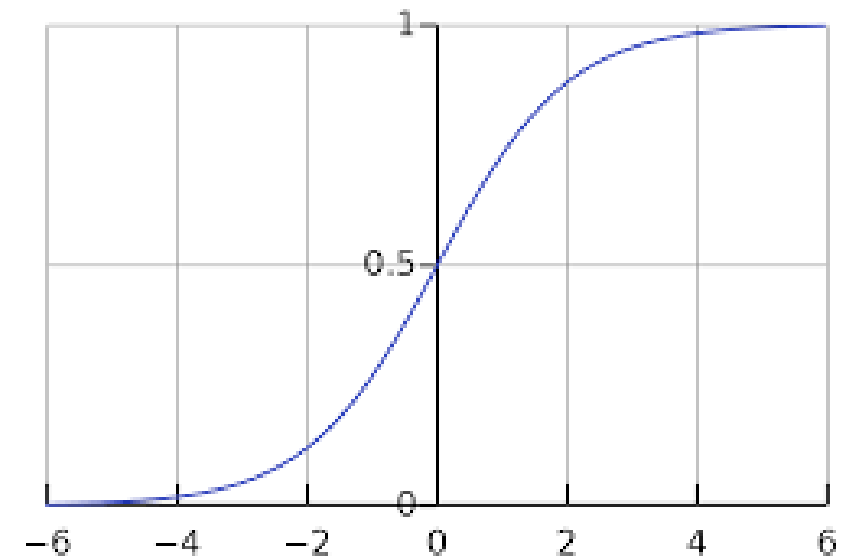
Logistic regression

$$h_{\theta}(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1+e^{-z}}$$

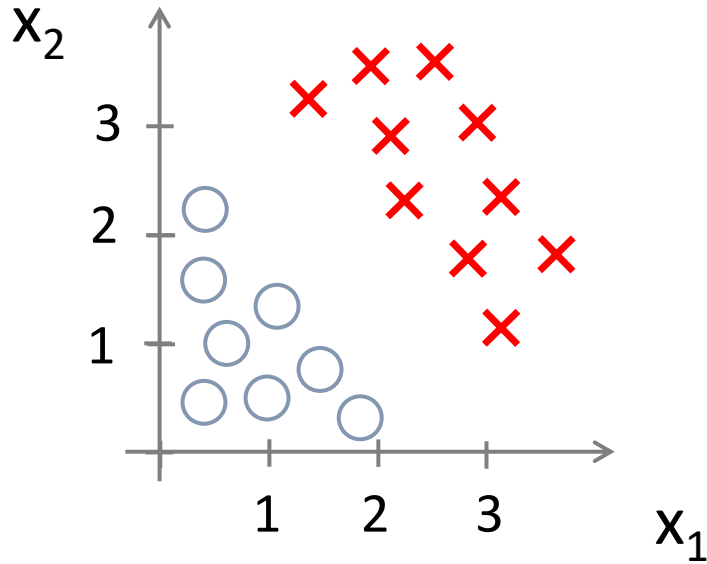
Suppose predict “ $y = 1$ ” if $h_{\theta}(x) \geq 0.5$

predict “ $y = 0$ ” if $h_{\theta}(x) < 0.5$



Let's use this to better understand how the hypothesis of logistic regression makes those predictions.

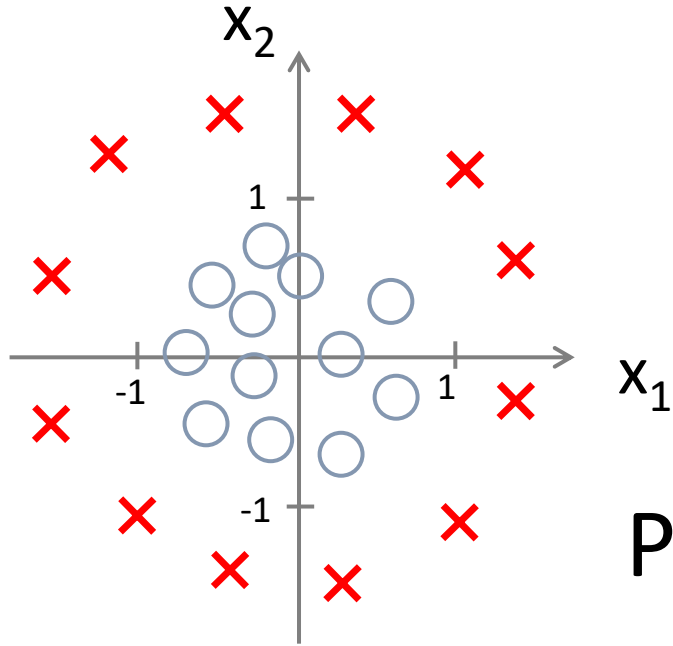
Decision Boundary



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

Predict “ $y = 1$ ” if $-3 + x_1 + x_2 \geq 0$

Non-linear decision boundaries



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

Predict “ $y = 1$ ” if $-1 + x_1^2 + x_2^2 \geq 0$

Now that we know what $h(x)$ can represent.

Next: see how to automatically choose the parameters θ .

So that given a training set we can automatically fit the parameters to our data.

Logistic Regression

Cost Function

Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

m examples $x \in \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_n \end{bmatrix} \quad x_0 = 1, y \in \{0, 1\}$

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

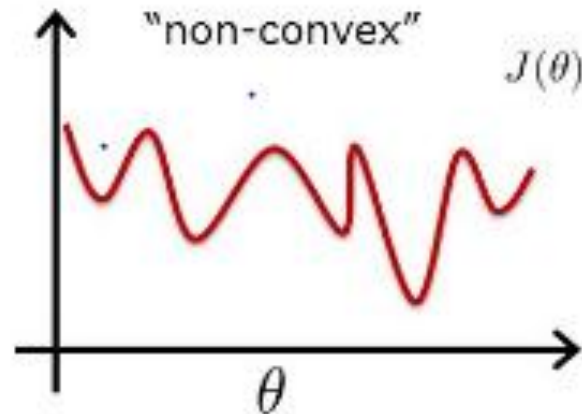
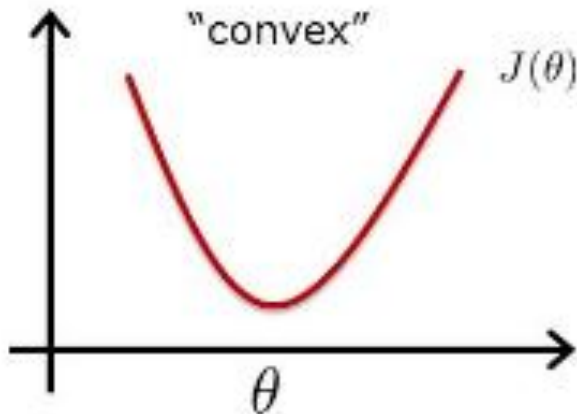
How to choose parameters θ ?

Cost function

Linear regression: $J(\theta) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2$

$$\text{Cost}(h_{\theta}(x^{(i)}), y^{(i)}) = \frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

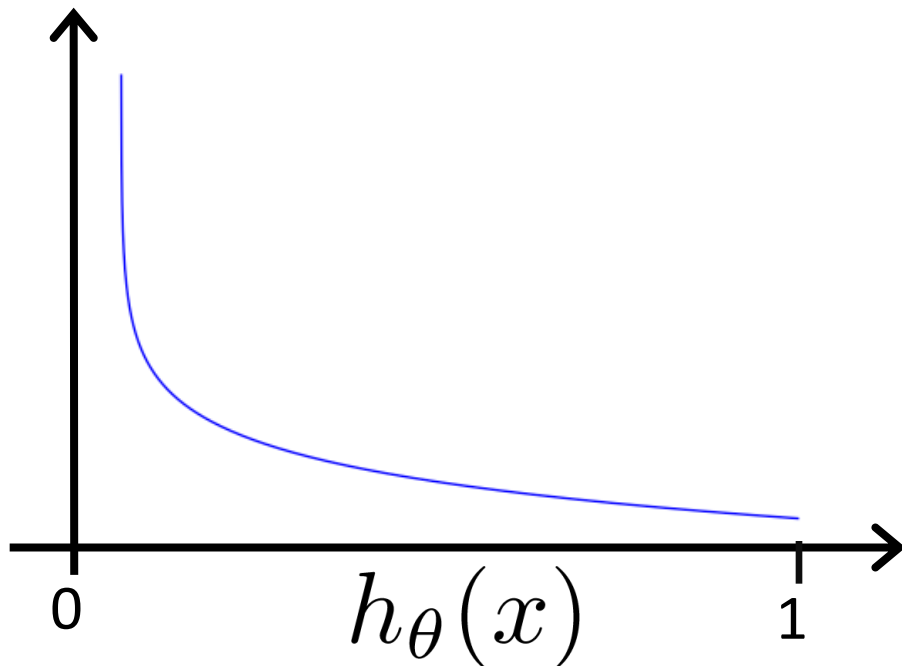
Convex Vs Non-Convex



Logistic regression cost function

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

If $y = 1$



Cost = 0 if $y = 1, h_{\theta}(x) = 1$

But as $h_{\theta}(x) \rightarrow 0$

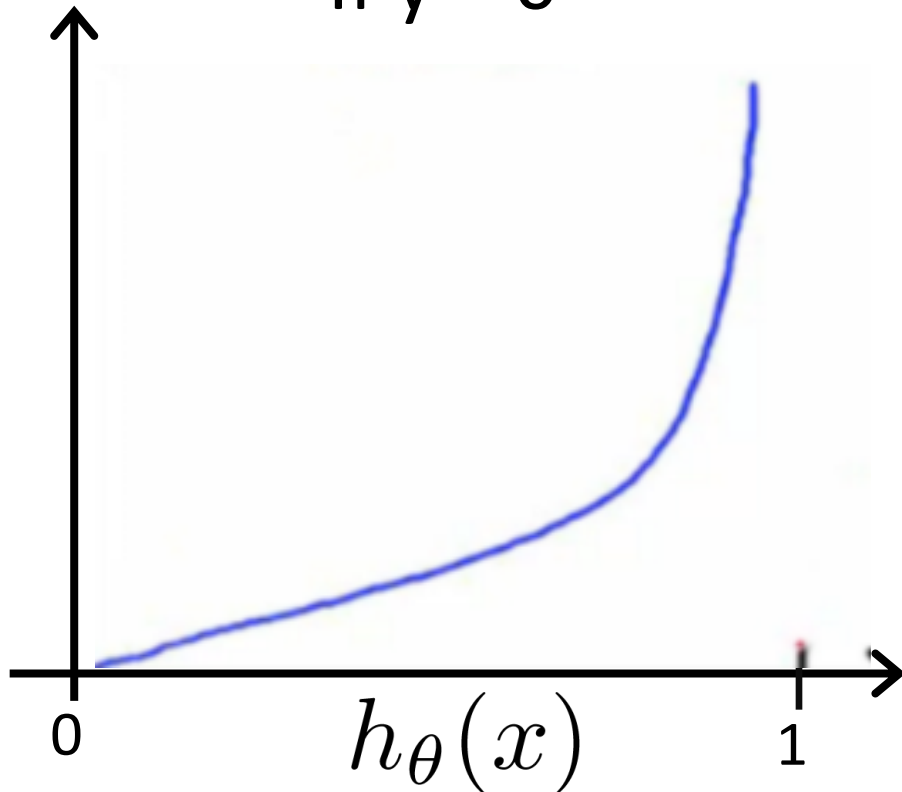
$\text{Cost} \rightarrow \infty$

Captures intuition that if $h_{\theta}(x) = 0$, (predict $P(y = 1|x; \theta) = 0$), but $y = 1$, we'll penalize learning algorithm by a very large cost.

Logistic regression cost function

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

If $y = 0$



Logistic Regression

Cost Function and Gradient Descent

Logistic regression cost function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

Note: $y = 0$ or 1 always

Logistic regression cost function

$$\begin{aligned} J(\theta) &= \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)}) \\ &= -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right] \end{aligned}$$

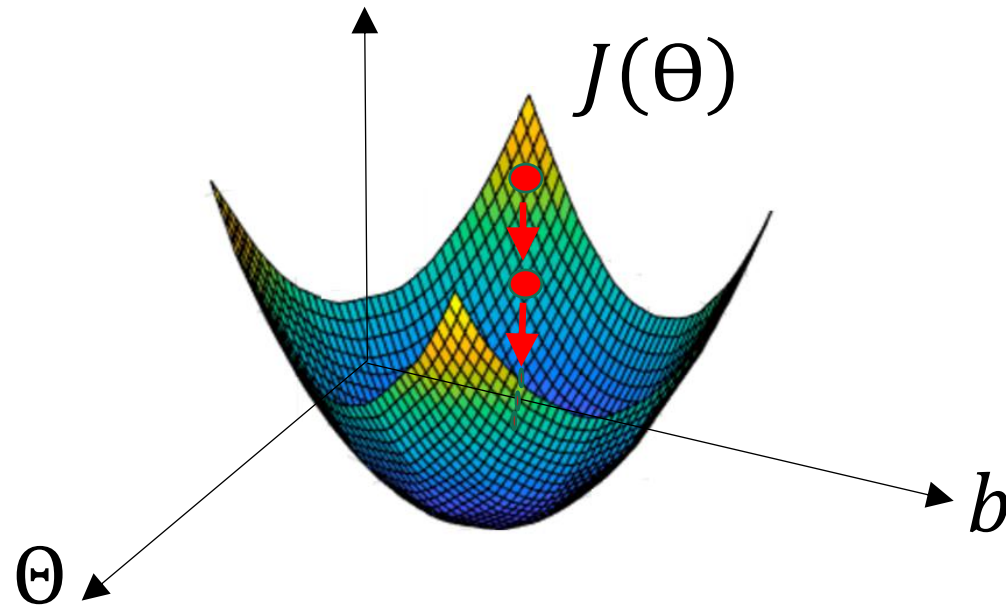
To fit parameters θ :

$$\min_{\theta} J(\theta)$$

To make a prediction given new x :

$$\text{Output } h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Gradient Descent



Gradient Descent

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right]$$

Want $\min_{\theta} J(\theta)$:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

}

(simultaneously update all θ_j)

Gradient Descent

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right]$$

Want $\min_{\theta} J(\theta)$:

Repeat {

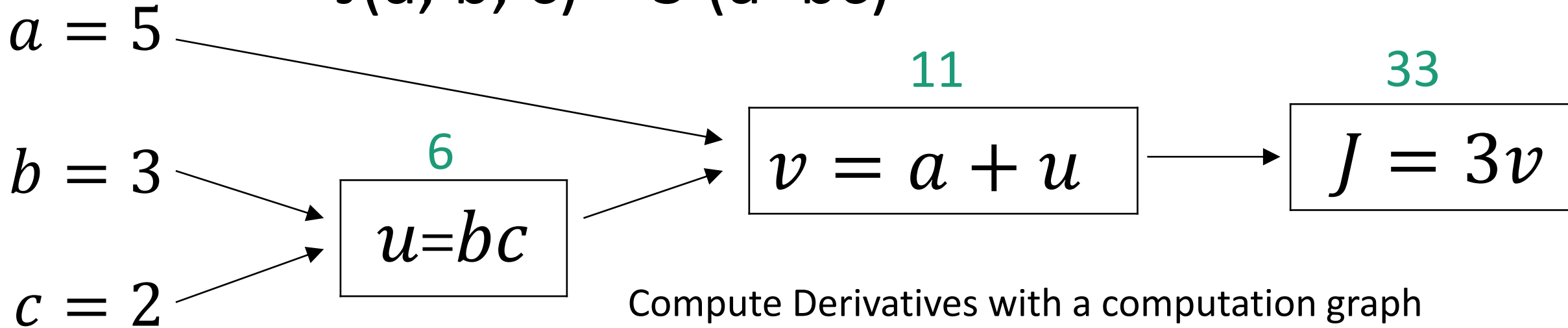
$$\theta_j := \theta_j - \alpha \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

} (simultaneously update all θ_j)

Algorithm looks identical to linear regression!

Computation Graph

$$J(a, b, c) = 3(a+bc)$$



Compute Derivatives with a computation graph

$$\frac{\partial J}{\partial u} = \frac{\partial J}{\partial v} * \frac{\partial v}{\partial u}$$

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial u} * \frac{\partial u}{\partial b}$$

$$\frac{\partial J}{\partial c} = \frac{\partial J}{\partial u} * \frac{\partial u}{\partial c}$$

$$\frac{\partial J}{\partial a} = \frac{\partial J}{\partial v} * \frac{\partial v}{\partial a}$$

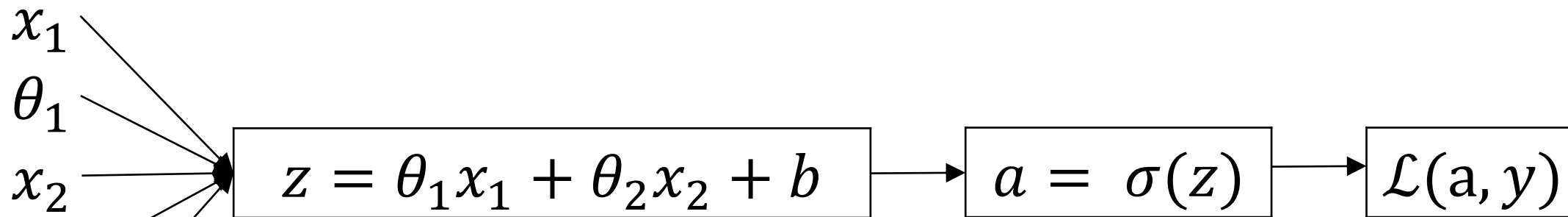
Computation Graph: Logistic Regression and Gradient

$$z = \theta^T x + b$$

$$\hat{y} = a = \sigma(z)$$

$$\mathcal{L}(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$$

Logistic regression derivatives



$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a} * \frac{\partial a}{\partial z} = a - y$$

$$\frac{\partial a}{\partial z} = a(1 - a)$$

$$\frac{\partial \mathcal{L}}{\partial a} = -\frac{y}{a} + \frac{1 - y}{1 - a}$$

Logistic regression on m examples

$$J(\theta, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(a^i, y^i)$$

$$a^i = \hat{y} = \sigma(z^i) = \sigma(\theta^T x^i + b)$$

$$\frac{\partial}{\partial \theta_i} J(\theta, b) = \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_i} \mathcal{L}(a^i, y^i)$$

Logistic regression on m examples

$J=0$; $d\Theta_1=0$; $d\Theta_2=0$; $db=0$

For $i=1$ to m

$$z^i = \Theta^T x^i + b$$

$$a^i = \sigma(z^i)$$

$$J += -[y^i \log(a^i) + (1 - y^i) \log(1 - a^i)]$$

$$dz^i = a^i - y^i$$

$$d\Theta_1 += x_1^i * dz^i$$

$$d\Theta_2 += x_2^i * dz^i$$

$$db += dz^i$$

$$J = J/m$$

$$d\Theta_1 /= m$$

$$d\Theta_2 /= m$$

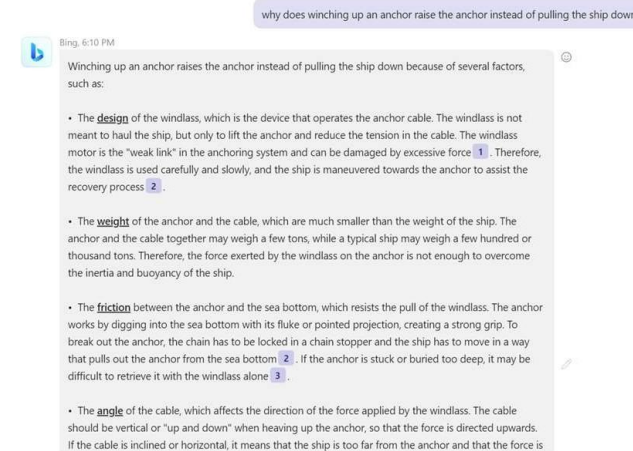
$$db /= m$$

Artificial Neural Networks

Neural networks have taken over AI

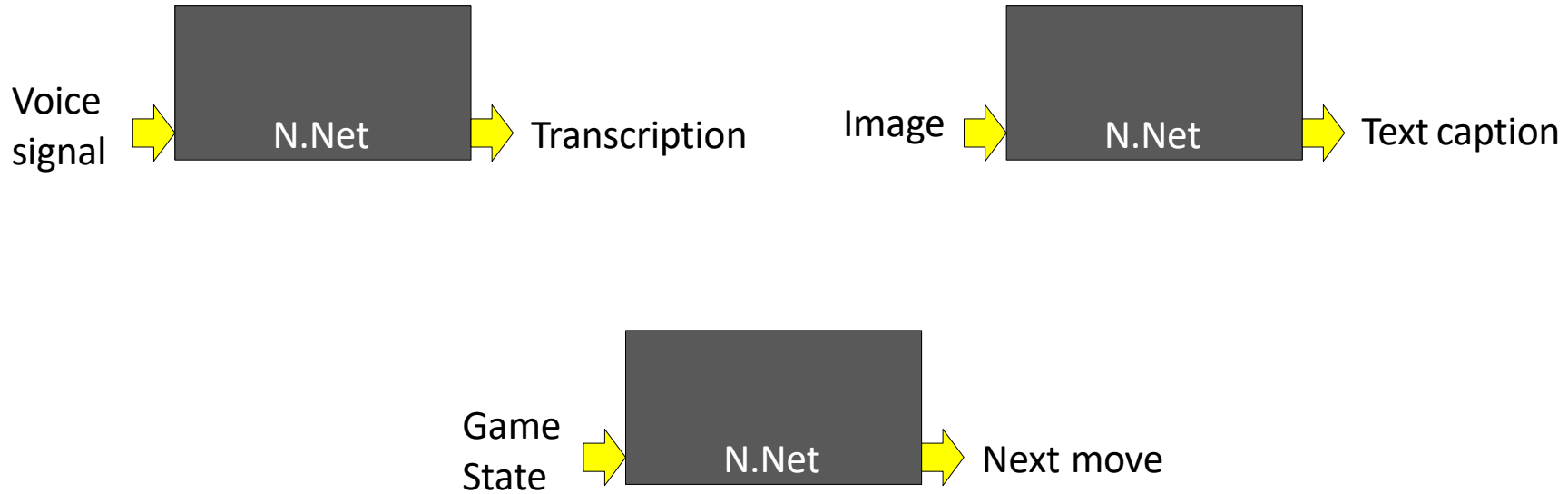


"A harlequin dancing in a field on a sunny beautiful day in a beau..."



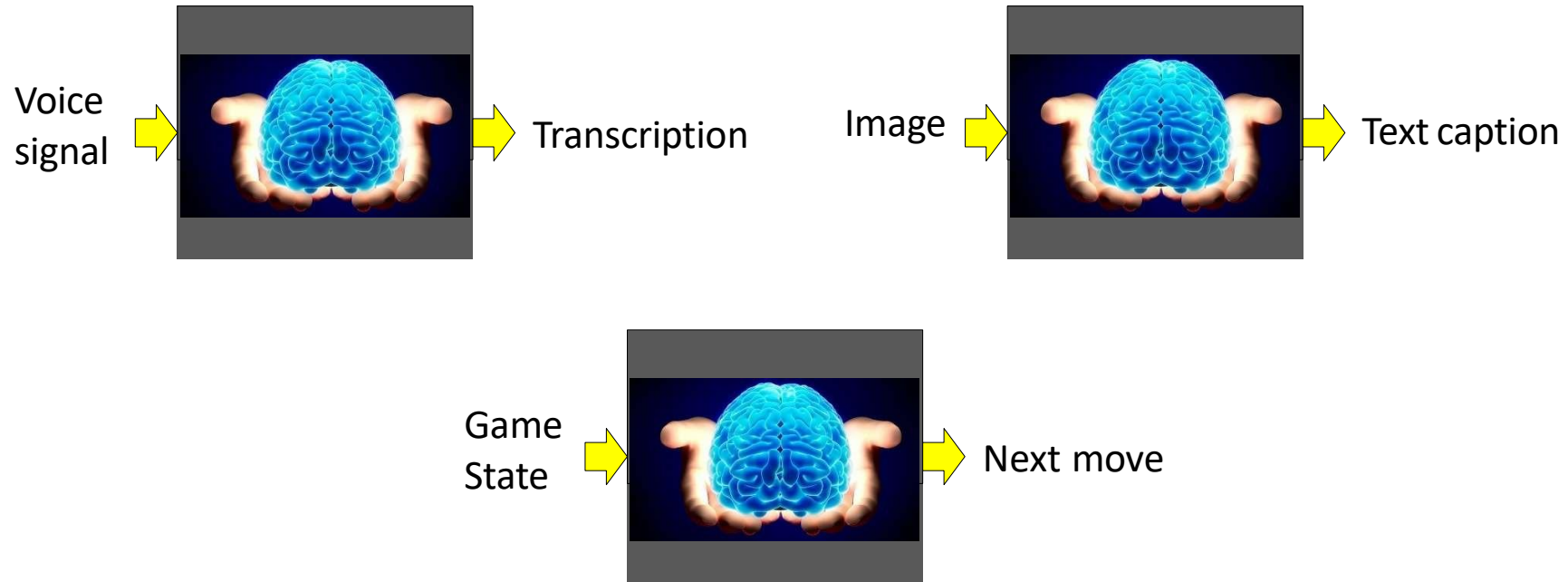
- Tasks that are made possible by NNs, aka deep learning
 - Tasks that were once assumed to be purely in the human domain of expertise

So what are neural networks??



- What are these boxes?
 - Functions that take an input and produce an output
 - What are these functions?

The human perspective



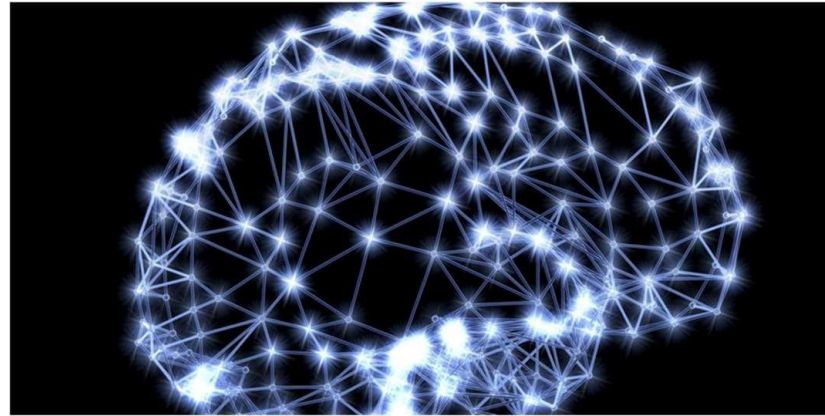
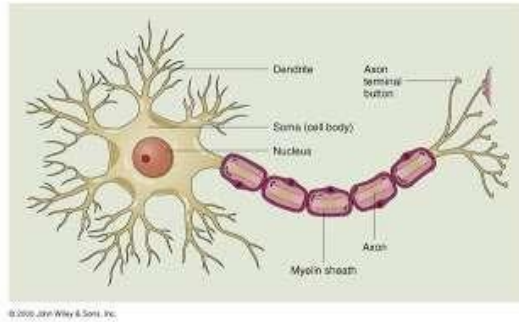
- In a human, those functions are computed by the brain...

NNets and the brain



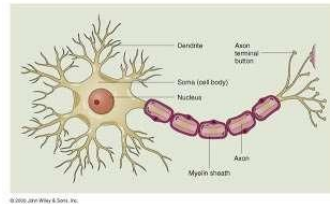
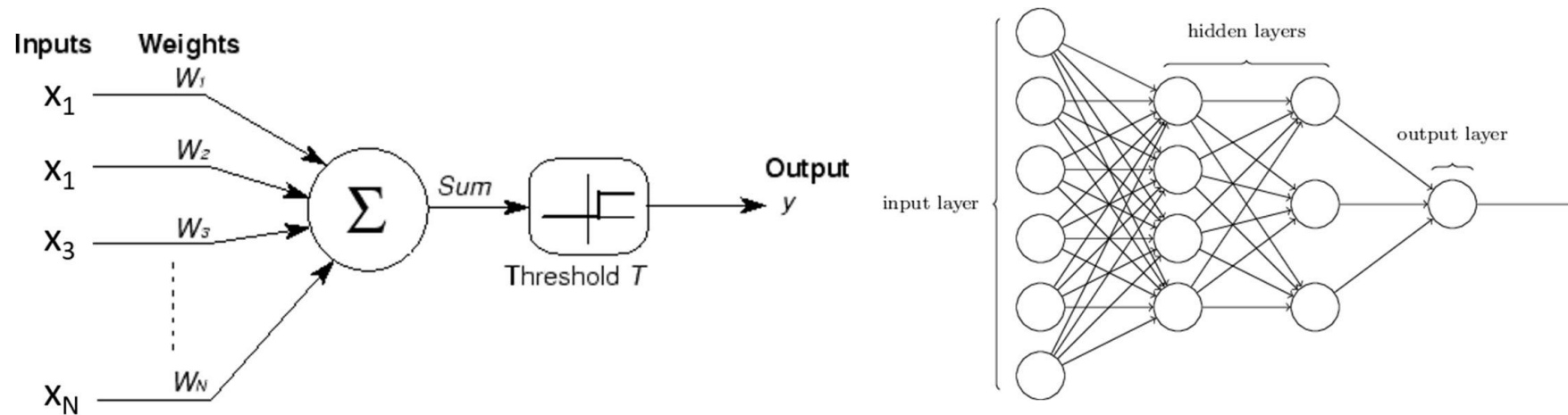
- In their basic form, NNets mimic the networked structure in the brain

The brain



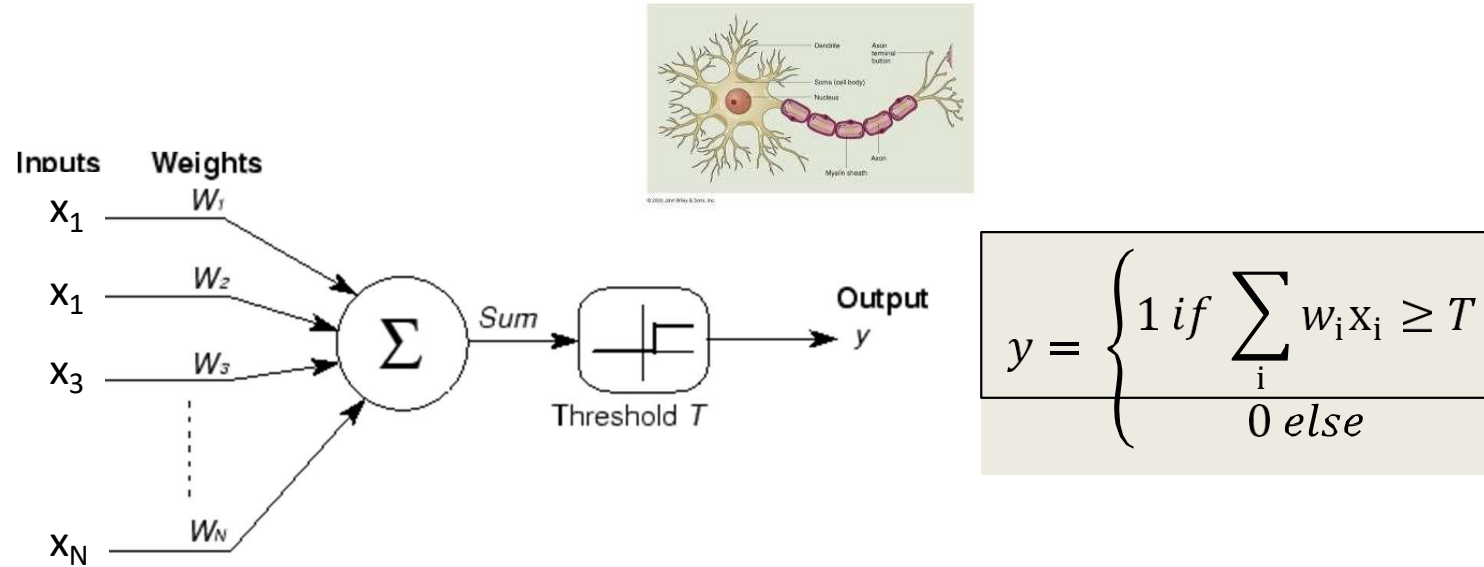
- The Brain is composed of networks of neurons

Nnets and the brain



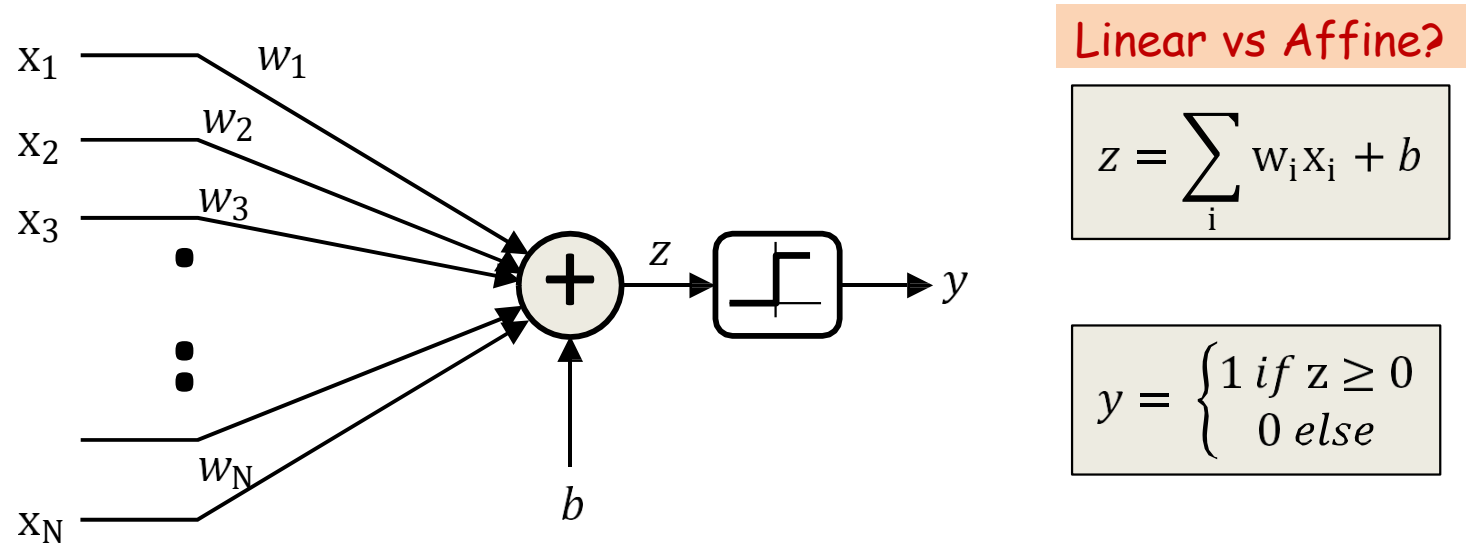
- Neural nets are composed of networks of computational models of neurons called perceptrons

The perceptron



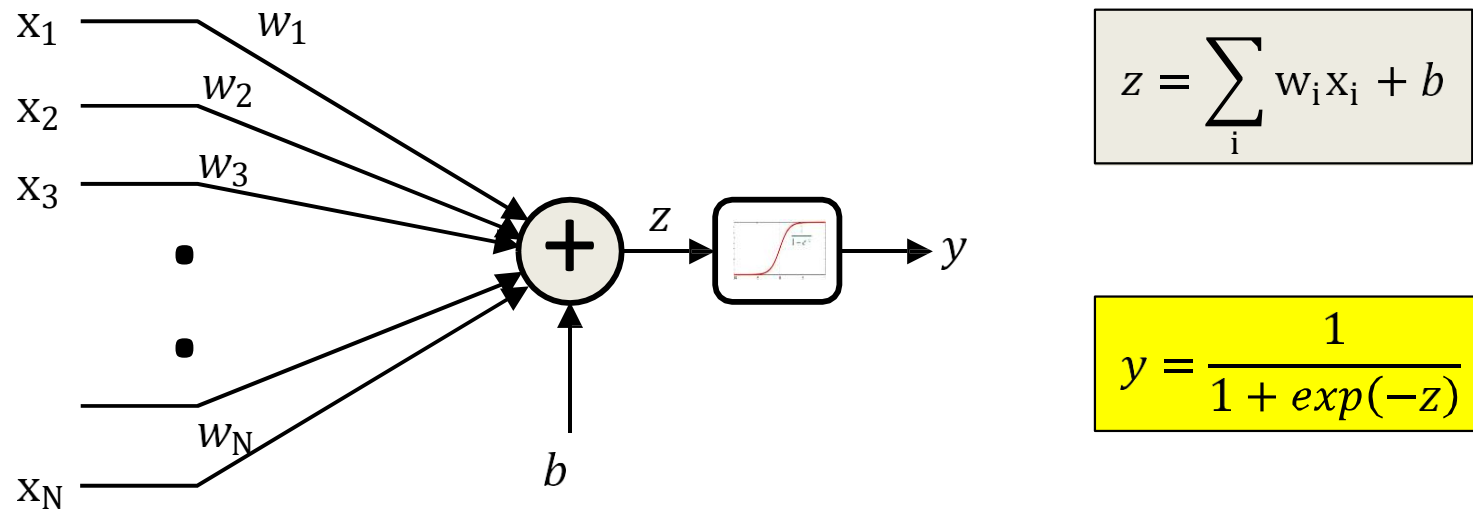
- A threshold unit
 - “Fires” if the weighted sum of inputs exceeds a threshold
 - Electrical engineers will call this a **threshold gate**
 - A basic unit of Boolean circuits

A better figure



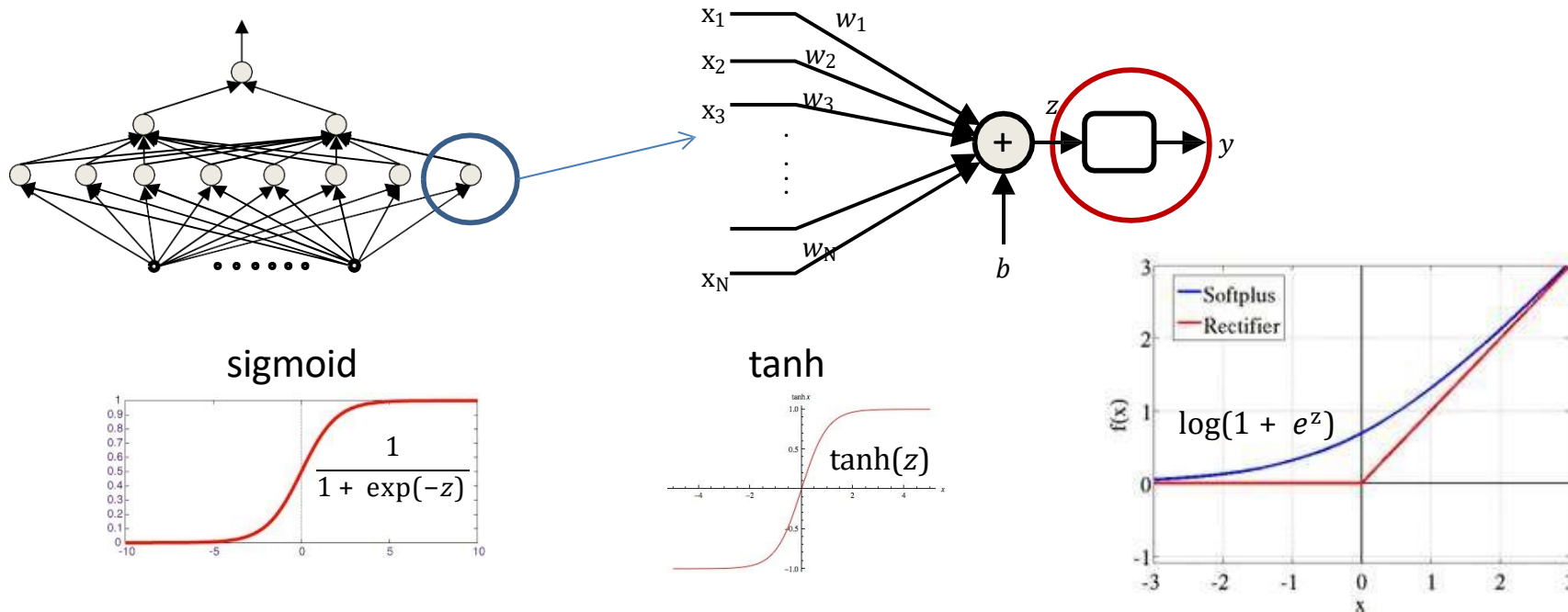
- A threshold unit
 - “Fires” if the affine function of inputs is positive
 - The bias is the negative of the threshold T in the previous slide

The “soft” perceptron (logistic)



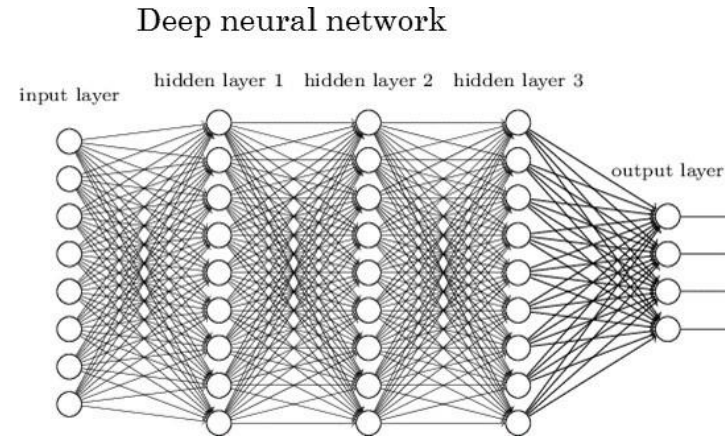
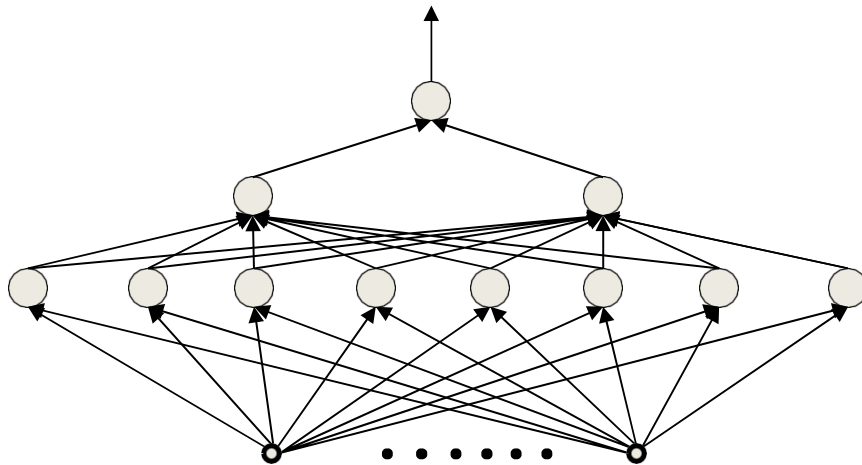
- A “squashing” function instead of a threshold at the output
 - The **sigmoid** “activation” replaces the threshold
 - **Activation:** The function that acts on the weighted combination of inputs (and bias)

Other “activations”



- Does not always have to be a squashing function
 - We will hear more about activations later
- We will continue to assume a “threshold” activation in this lecture

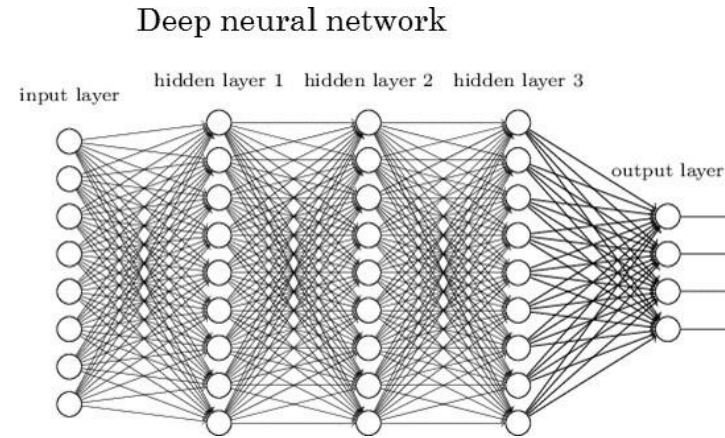
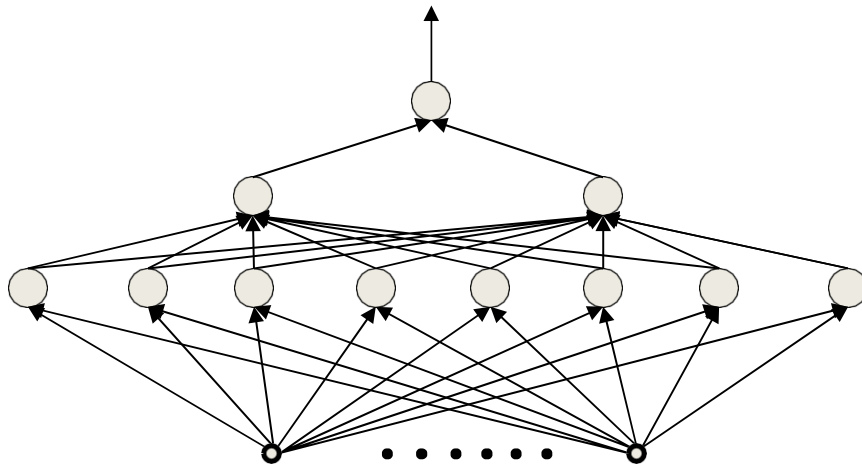
The *multi-layer* perceptron



- A network of perceptrons
 - Perceptrons “feed” other perceptrons
 - We give you the “formal” definition of a layer shortly



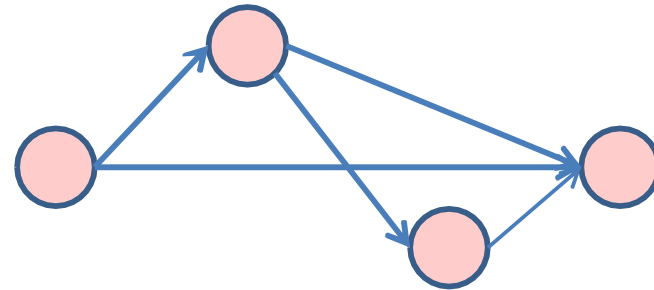
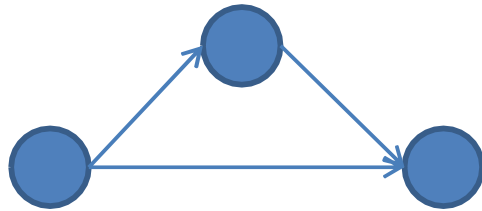
Defining “depth”



- What is a “deep” network

Deep Structures

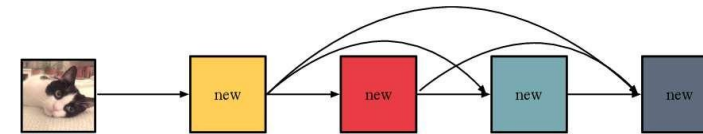
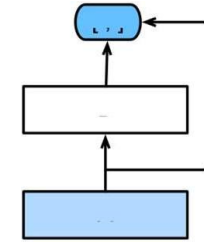
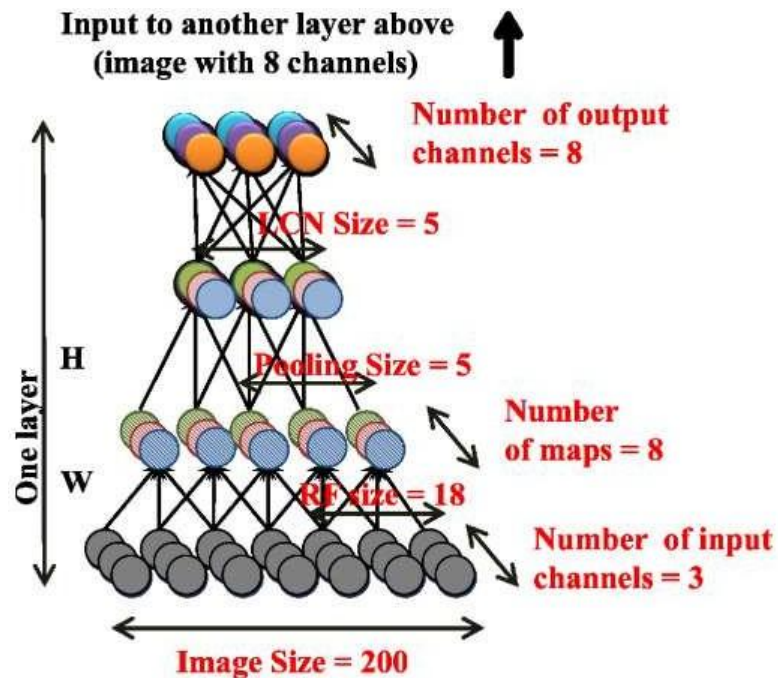
- In any directed graph with input source nodes and output sink nodes, “depth” is the length of the longest path from a source to a sink
 - A “source” node in a directed graph is a node that has only outgoing edges
 - A “sink” node is a node that has only incoming edges



- Left: Depth = 2. Right: Depth = 3

Deep Structures

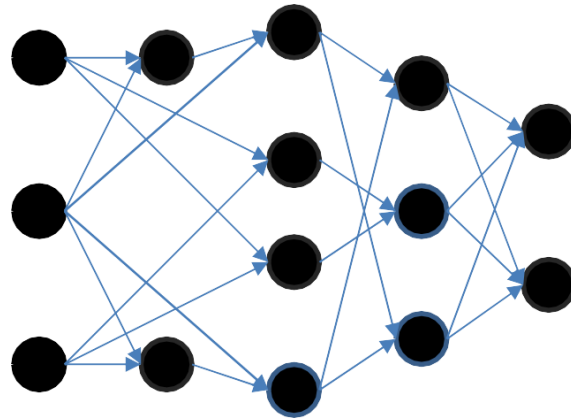
- *Deep structure*
 - The input is the “source”,
 - The output nodes are “sinks”



- “Deep” → Depth of output neurons is greater than 2

What is a layer?

- A “layer” is the set of neurons that are all at the same depth with respect to the input (source)
 - “Depth” of a layer – the depth of the neurons in the layer w.r.t. input

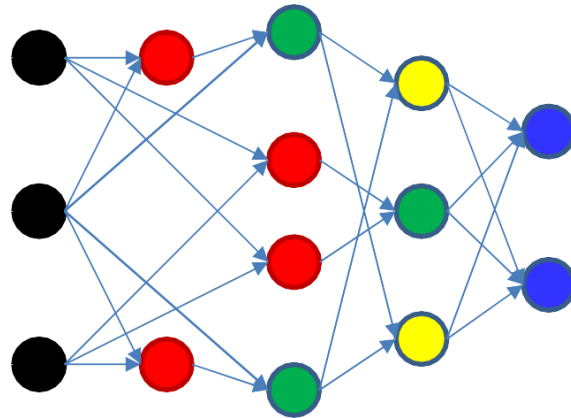


Input:
Layer 1:
Layer 2:
Layer 3:
Layer 4:

- “Deep” → At least 3 layers
 - Output layer depth is at least 3

What is a layer?

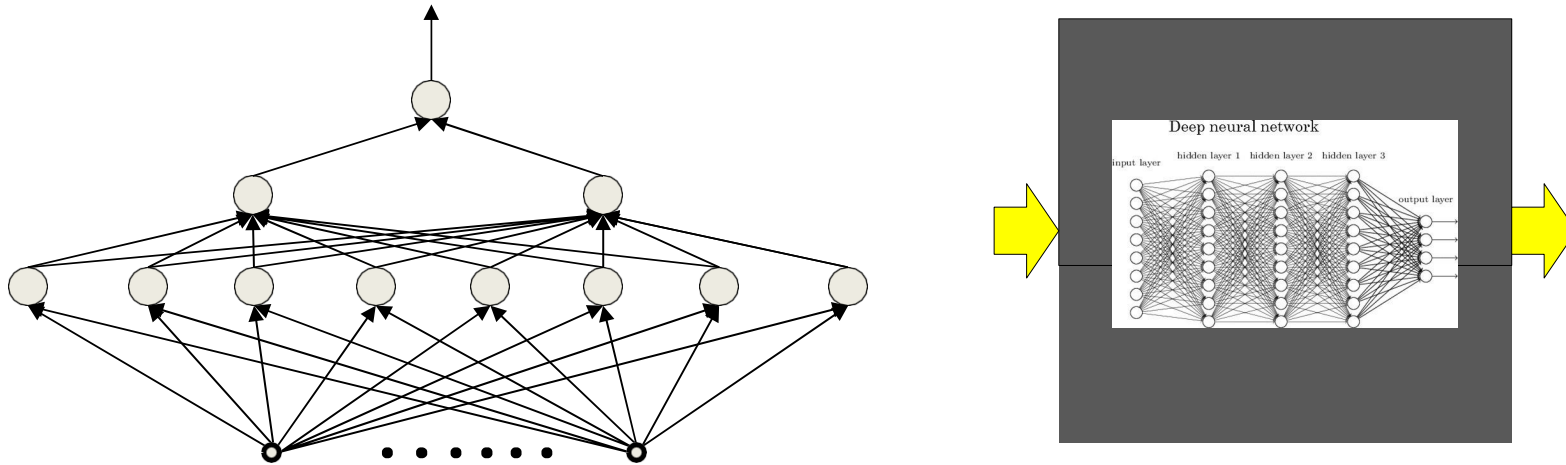
- A “layer” is the set of neurons that are all at the same depth with respect to the input (source)
 - “Depth” of a layer – the depth of the neurons in the layer w.r.t. input



Input: Black
Layer 1: Red
Layer 2: Green
Layer 3: Yellow
Layer 4: Blue

- “Deep” → At least 3 layers
 - Output layer depth is at least 3

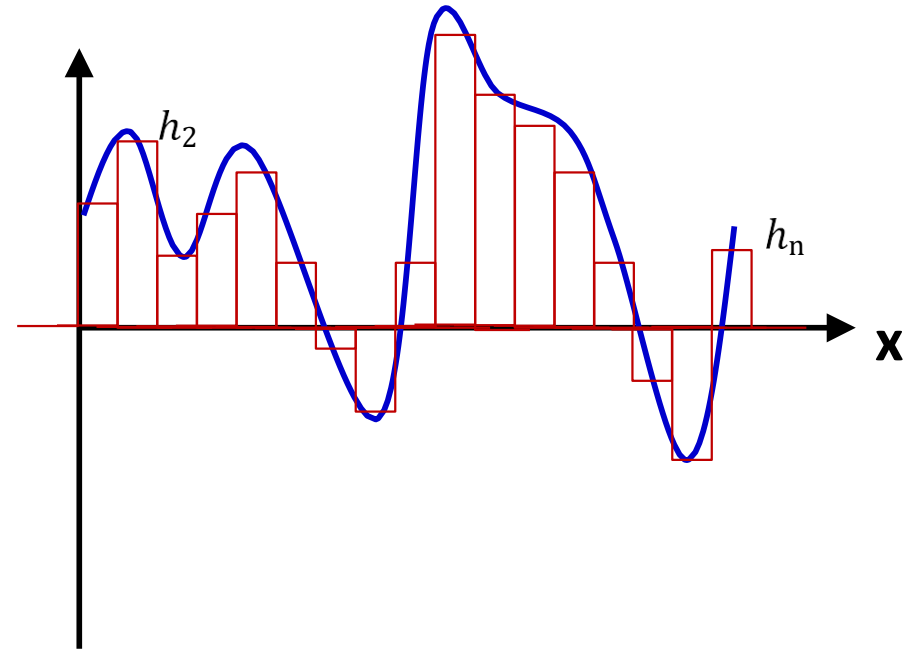
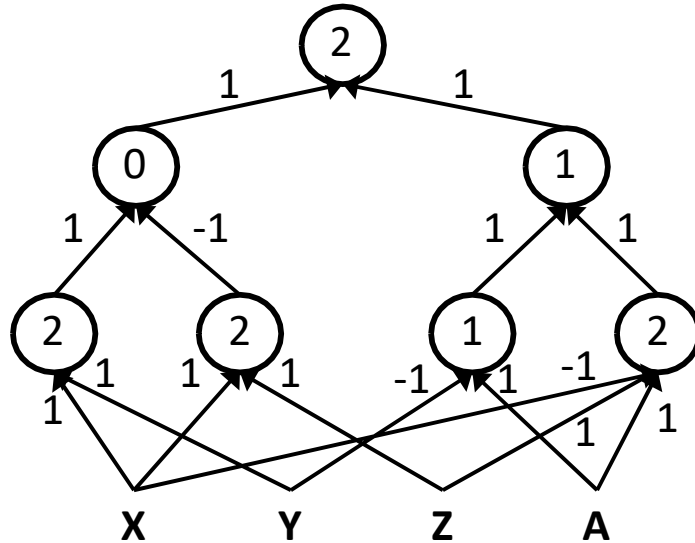
The multi-layer perceptron



- Inputs are real or Boolean stimuli
- Outputs are real or Boolean values
 - Can have multiple outputs for a single input
- **What can this network compute?**
 - **What kinds of input/output relationships can it model?**

MLPs approximate functions

$$((A \& \bar{X} \& Z) | (A \& \bar{Y})) \& ((X \& Y) | \overline{(X \& Z)})$$



- MLPs can compose Boolean functions
- MLSs can compose category functions (classifiers)
- MLPs can compose real-valued functions
- What are the limitations?

Today

- Multi-layer Perceptrons as universal Boolean functions
 - The need for depth
- MLPs as universal classifiers
 - The need for depth
- MLPs as universal approximators

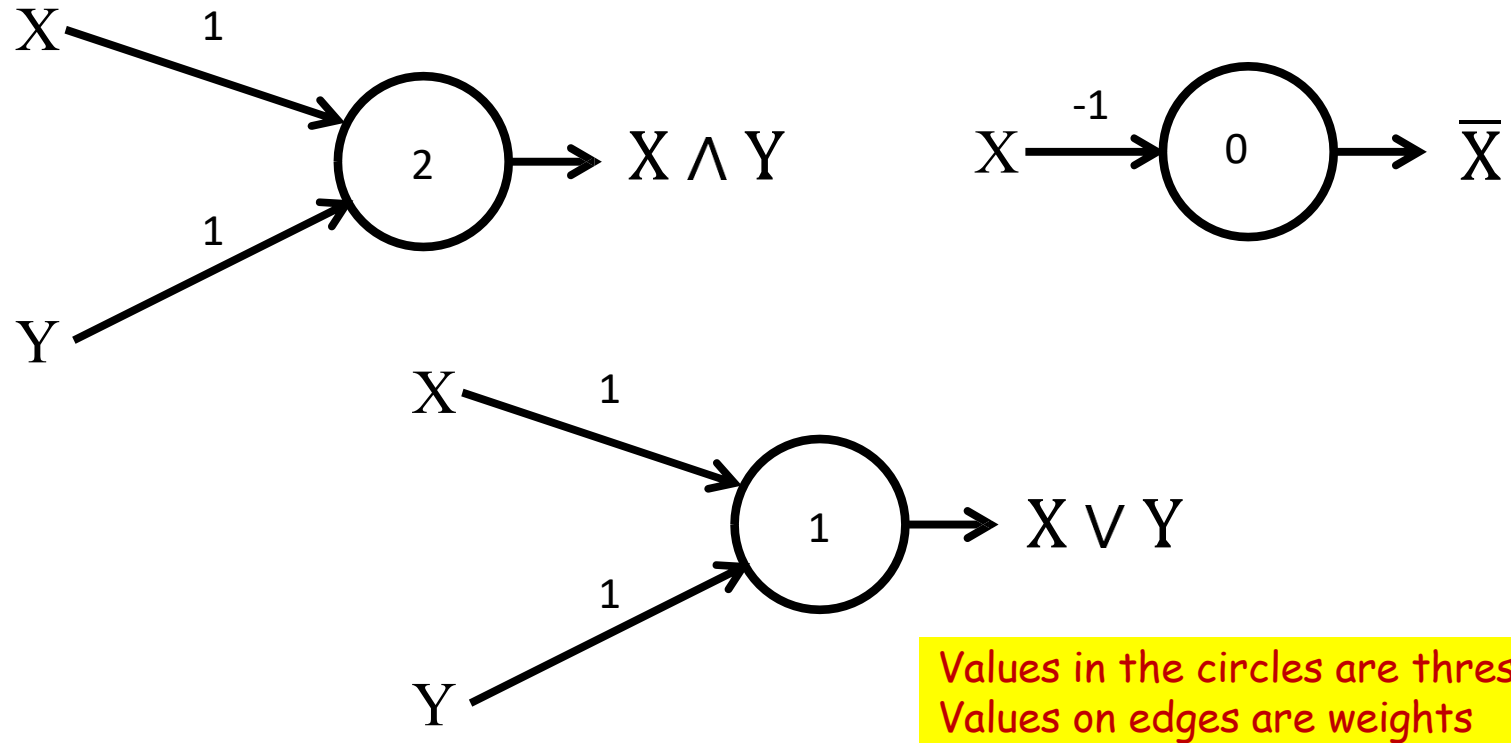
Today

- Multi-layer Perceptrons as universal Boolean functions
 - The need for depth
- MLPs as universal classifiers
 - The need for depth
- MLPs as universal approximators

The MLP as a Boolean function

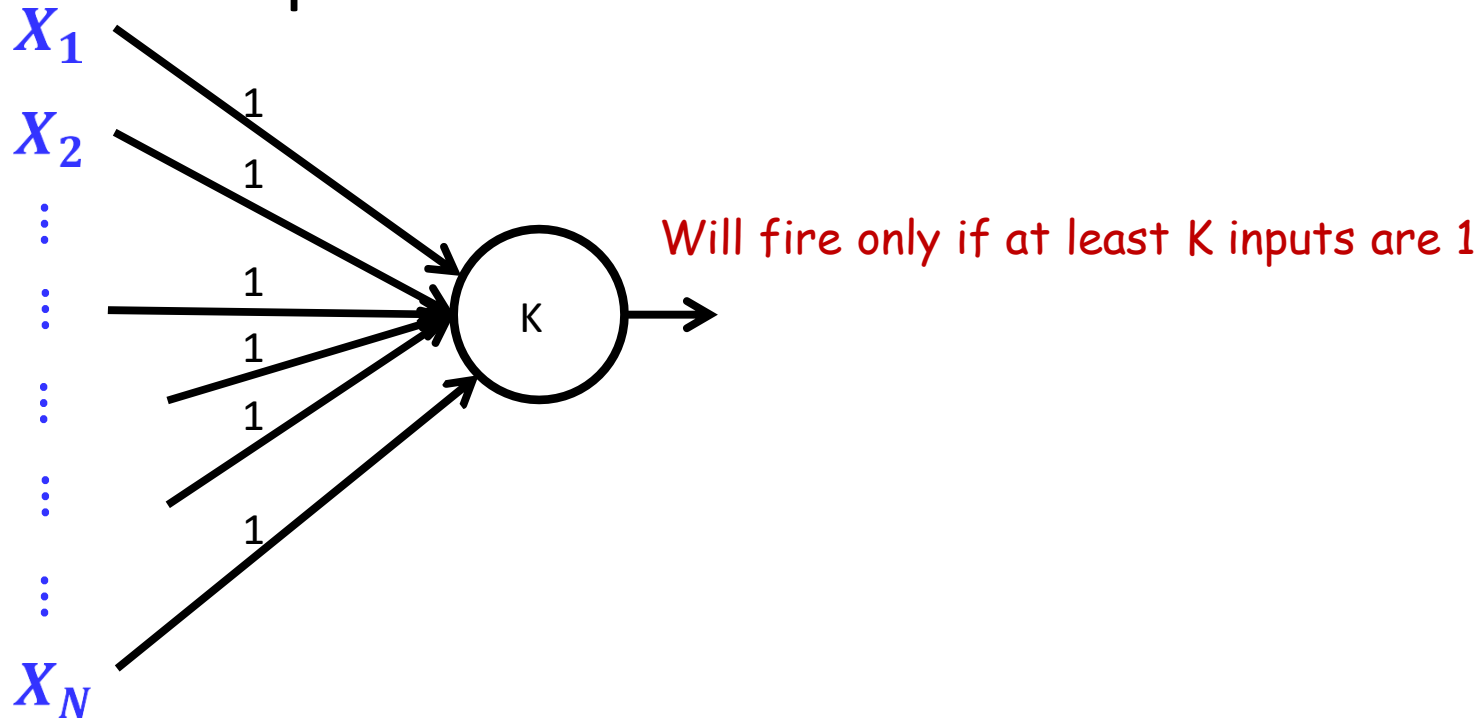
- How well do MLPs model Boolean functions?

The perceptron as a Boolean gate



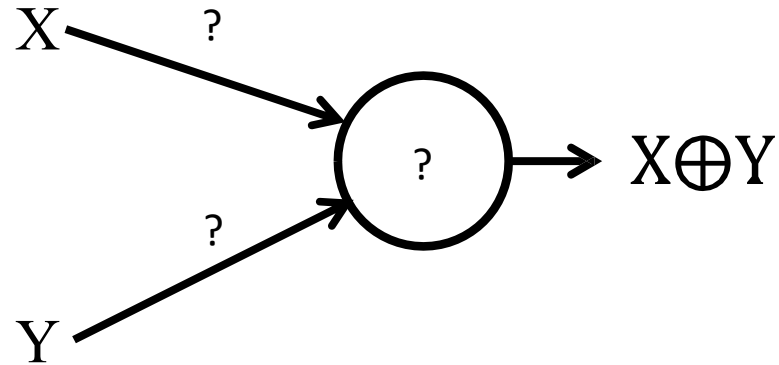
- A perceptron can model any simple binary Boolean gate

Perceptron as a Boolean Gate



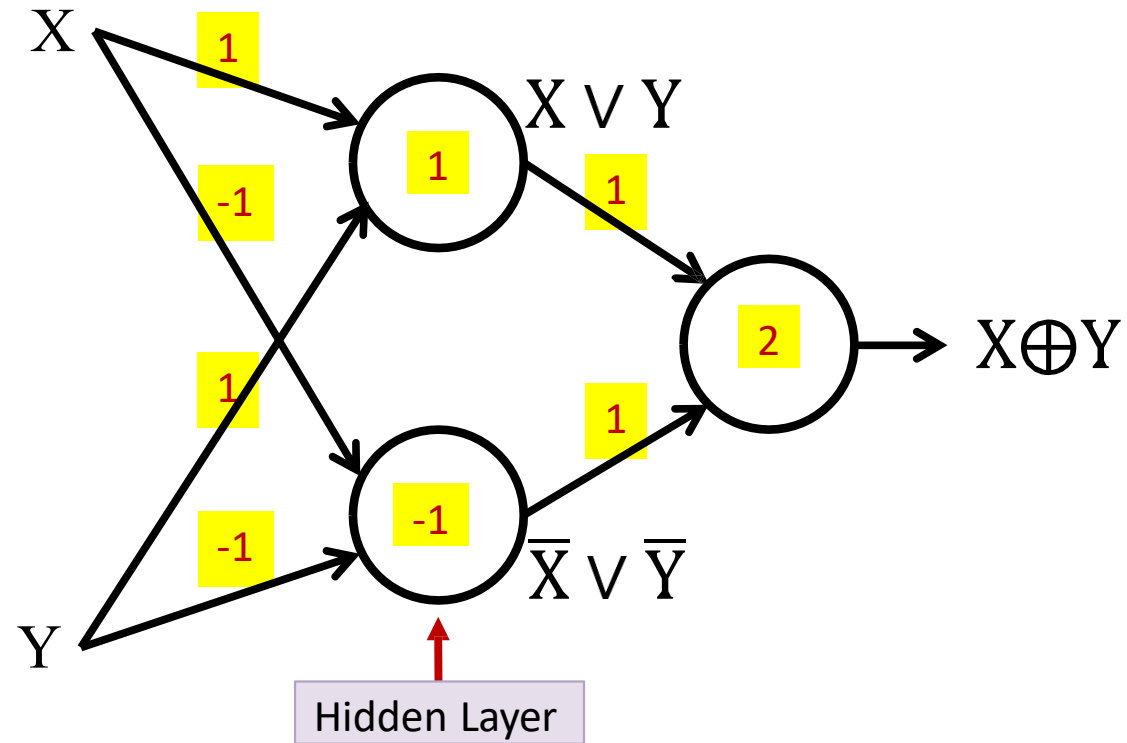
- Generalized *majority* gate
 - Fire if at least K inputs are of the desired polarity

The perceptron is not enough



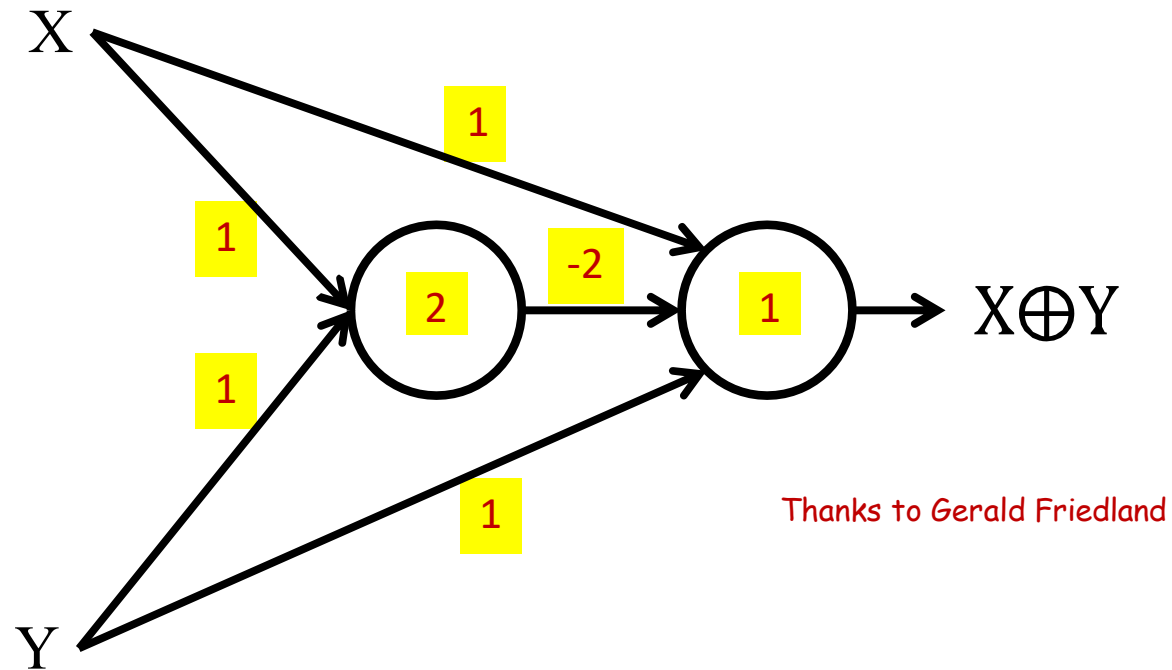
- Cannot compute an XOR

Multi-layer perceptron



- MLPs can compute the XOR

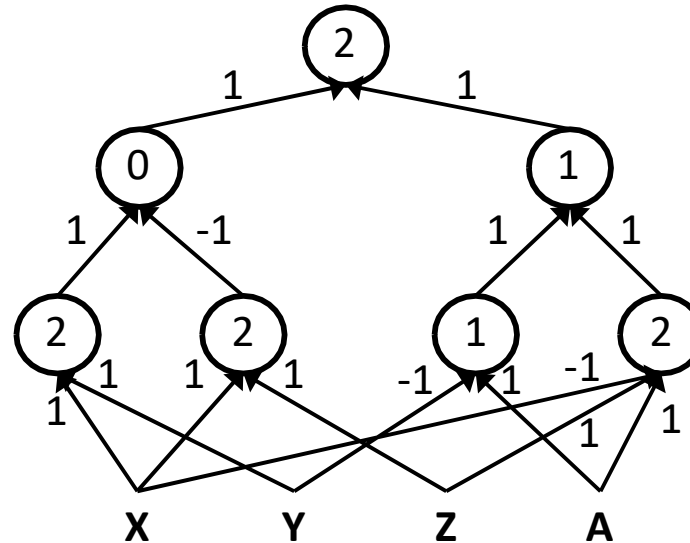
Multi-layer perceptron XOR



- With 2 neurons
 - 5 weights and two thresholds

Multi-layer perceptron

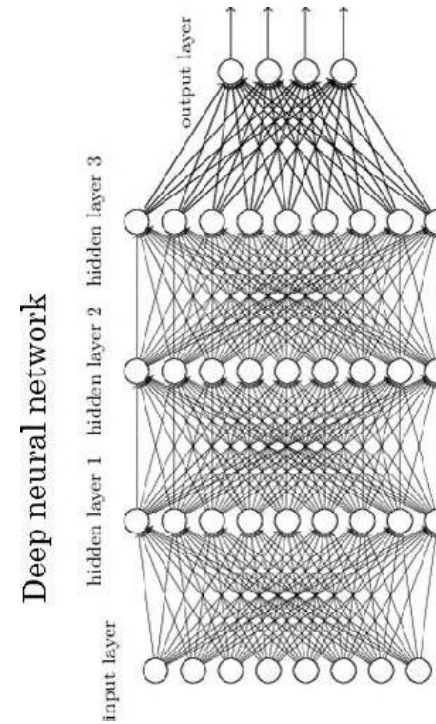
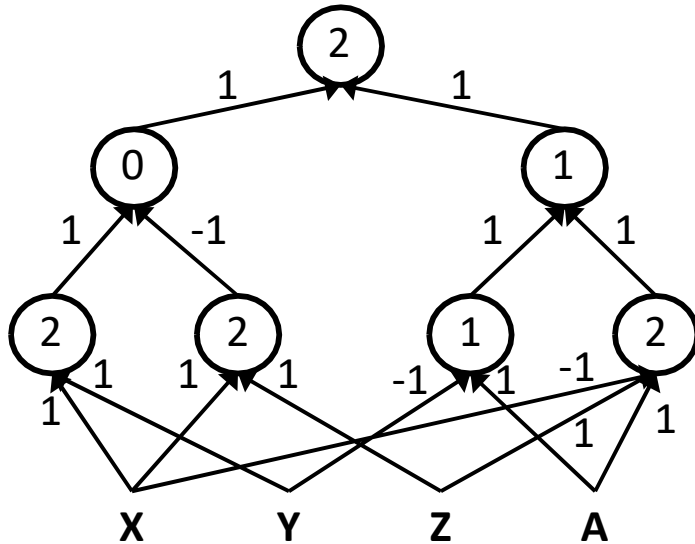
$$((A \& \bar{X} \& Z) | (A \& \bar{Y})) \& ((X \& Y) | (\bar{X} \& \bar{Z}))$$



- MLPs can compute more complex Boolean functions
- MLPs can compute *any* Boolean function
 - Since they can emulate individual gates
- **MLPs are *universal Boolean functions***

MLP as Boolean Functions

$$((A \& \bar{X} \& Z) \mid (A \& \bar{Y})) \& ((X \& Y) \mid \overline{(X \& Z)})$$



- MLPs are universal Boolean functions
 - Any function over any number of inputs and any number of outputs
- But how many “layers” will they need?

How many layers for a Boolean MLP?

Truth table shows *all* input combinations
for which output is 1

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

- ***A Boolean function is just a truth table***

How many layers for a Boolean MLP?

Truth table shows *all* input combinations
for which output is 1

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

$$Y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + \\ X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$

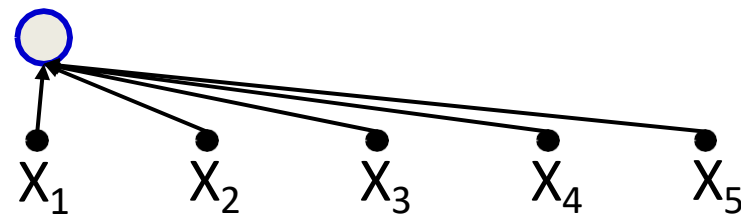
- Expressed in disjunctive normal form

How many layers for a Boolean MLP?

Truth table shows *all* input combinations for which output is 1

Truth Table					
X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

$$Y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1X_2X_3X_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



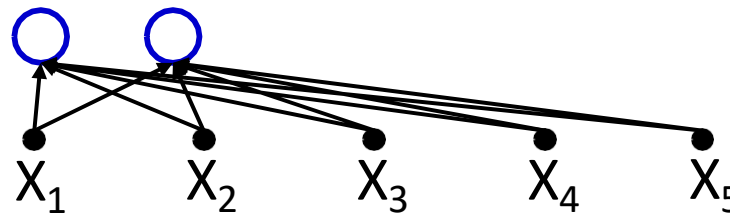
- Expressed in disjunctive normal form

How many layers for a Boolean MLP?

Truth table shows *all* input combinations
for which output is 1

Truth Table					
X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

$$Y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



- Expressed in disjunctive normal form

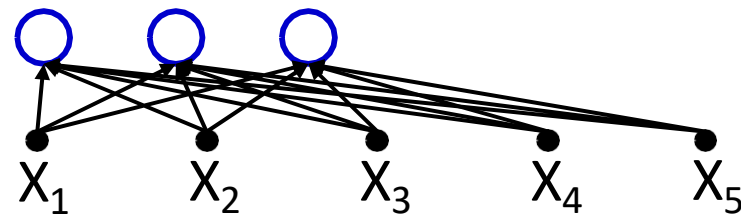
How many layers for a Boolean MLP?

Truth table shows *all* input combinations for which output is 1

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

$$Y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3X_4X_5$$



- Expressed in disjunctive normal form

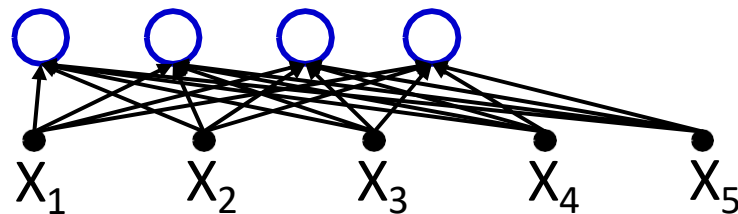
How many layers for a Boolean MLP?

Truth table shows *all* input combinations
for which output is 1

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

$$Y = \bar{X}_1 \bar{X}_2 X_3 X_4 \bar{X}_5 + \bar{X}_1 X_2 \bar{X}_3 X_4 X_5 + \bar{X}_1 X_2 X_3 \bar{X}_4 \bar{X}_5 + \bar{X}_1 \bar{X}_2 \bar{X}_3 \bar{X}_4 X_5 + X_1 \bar{X}_2 X_3 X_4 X_5 + X_1 X_2 \bar{X}_3 \bar{X}_4 X_5$$



- Expressed in disjunctive normal form

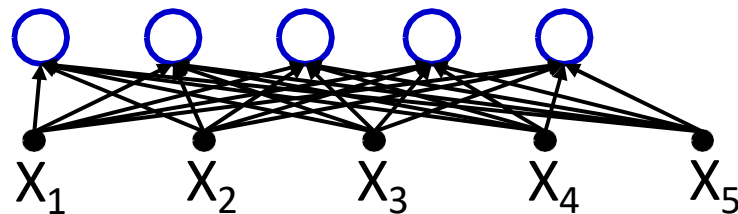
How many layers for a Boolean MLP?

Truth table shows *all* input combinations
for which output is 1

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

$$Y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + \bar{X}_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



- Expressed in disjunctive normal form

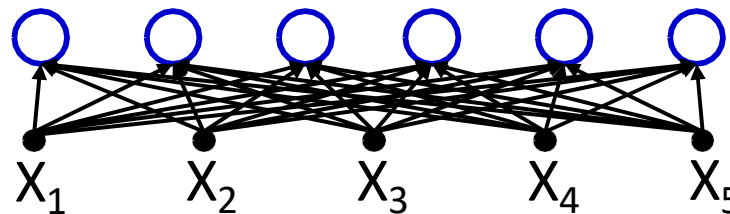
How many layers for a Boolean MLP?

Truth table shows *all* input combinations
for which output is 1

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

$$Y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + \bar{X}_1X_2\bar{X}_3\bar{X}_4X_5$$



- Expressed in disjunctive normal form

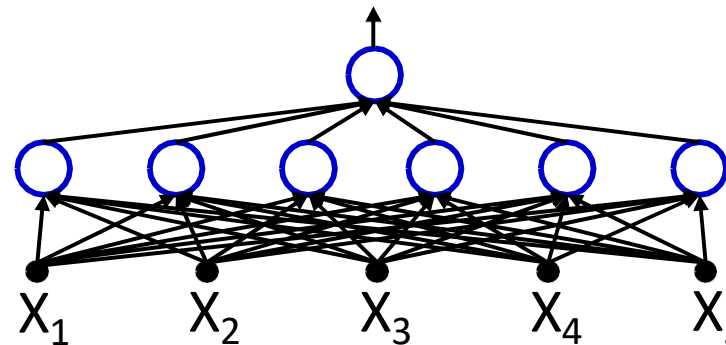
How many layers for a Boolean MLP?

Truth table shows *all* input combinations
for which output is 1

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

$$Y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



- Expressed in disjunctive normal form

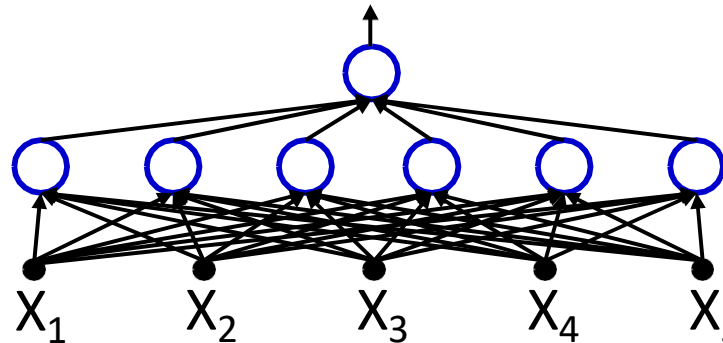
How many layers for a Boolean MLP?

Truth table shows *all* input combinations
for which output is 1

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

$$Y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



- *Any truth table can be expressed in this manner!*
- **A one-hidden-layer MLP is a Universal Boolean Function**

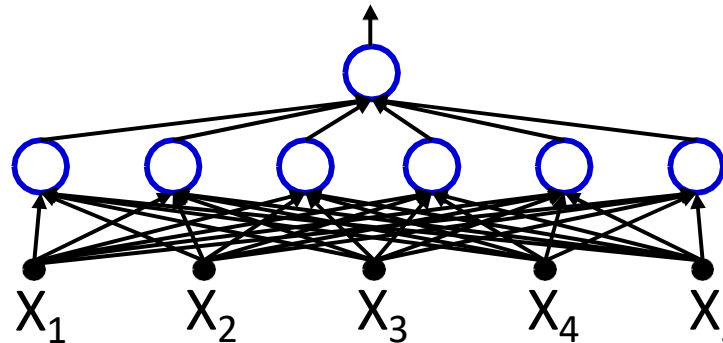
How many layers for a Boolean MLP?

Truth table shows *all* input combinations
for which output is 1

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

$$Y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



- *Any truth table can be expressed in this manner!*
- **A one-hidden-layer MLP is a Universal Boolean Function**

But what is the largest number of perceptrons required in the single hidden layer for an N-input-variable function?

Reducing a Boolean Function

WX \ YZ	00	01	11	10
00	1	0	0	1
01	1	1	0	0
11	1	0	0	0
10	1	0	0	1

This is a "Karnaugh Map"

It represents a truth table as a grid
Filled boxes represent input combinations
for which output is 1; blank boxes have
output 0

Adjacent boxes can be "grouped" to
reduce the complexity of the DNF formula
for the table

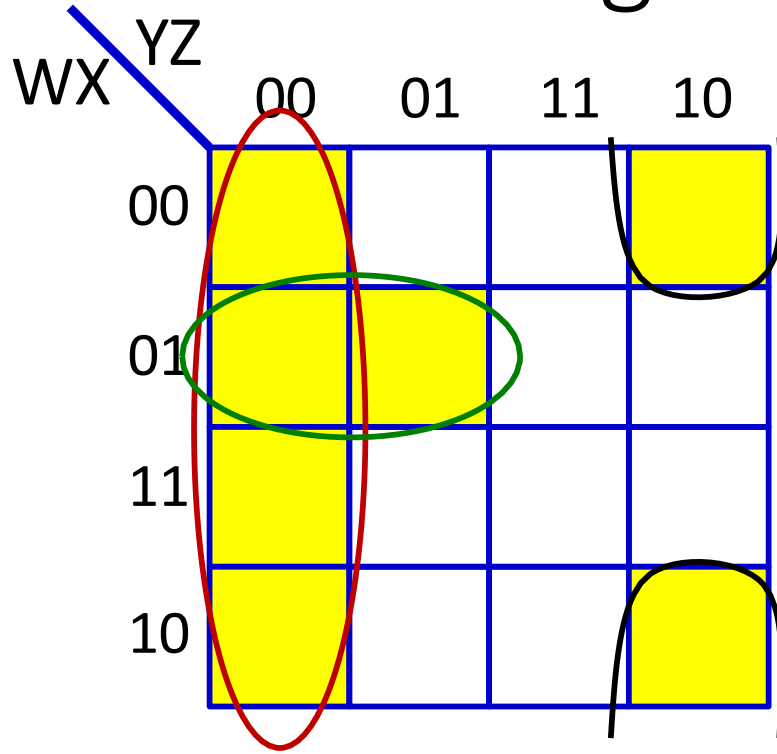
- DNF form:
 - Find groups
 - Express as reduced DNF

Reducing a Boolean Function

WX \ YZ				
	00	01	11	10
00				
01				
11				
10				

Basic DNF formula will require 7 terms

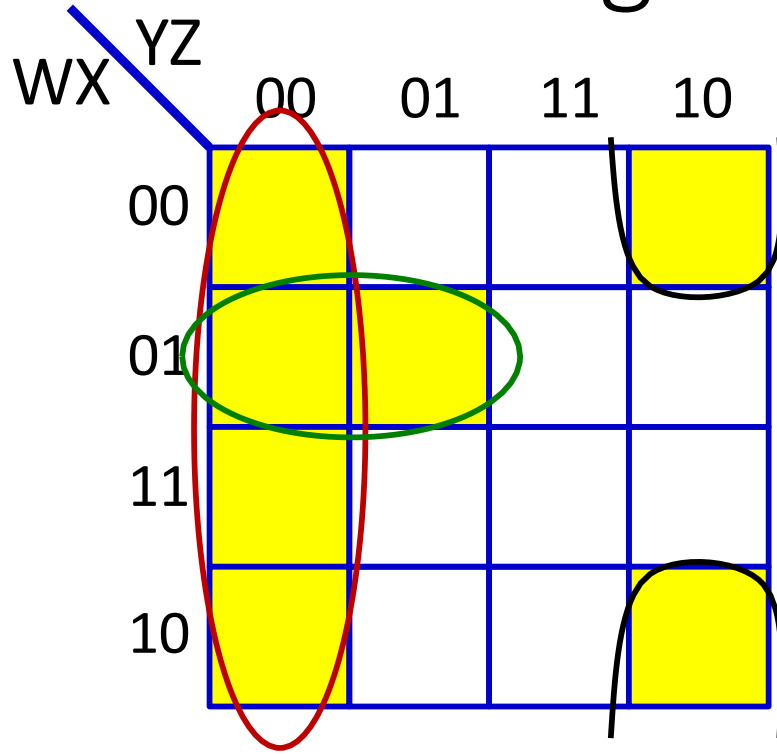
Reducing a Boolean Function



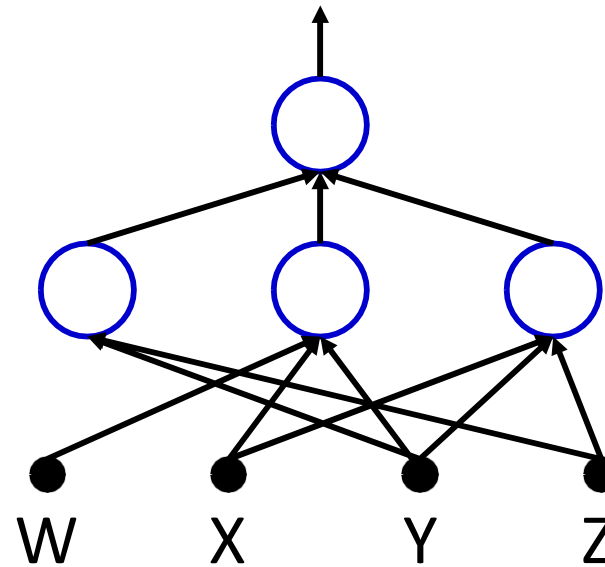
$$O = \bar{Y}\bar{Z} + \bar{W}X\bar{Y} + \bar{X}Y\bar{Z}$$

- *Reduced* DNF form:
 - Find groups
 - Express as reduced DNF

Reducing a Boolean Function



$$O = \bar{Y}\bar{Z} + \bar{W}X\bar{Y} + \bar{X}Y\bar{Z}$$



- *Reduced* DNF form:
 - Find groups
 - Express as *reduced* DNF
 - Boolean network for this function needs only 3 hidden units
 - Reduction of the DNF reduces the size of the one-hidden-layer network

Largest irreducible DNF?

		YZ			
		00	01	11	10
WX	00				
	01				
	11				
	10				

- What arrangement of ones and zeros simply cannot be reduced further?

Largest irreducible DNF?

		YZ			
		00	01	11	10
WX	00	Red	White	Red	White
	01	White	Red	White	Red
	11	Red	White	Red	White
	10	White	Red	White	Red

Red=0, white=1

- What arrangement of ones and zeros simply cannot be reduced further?

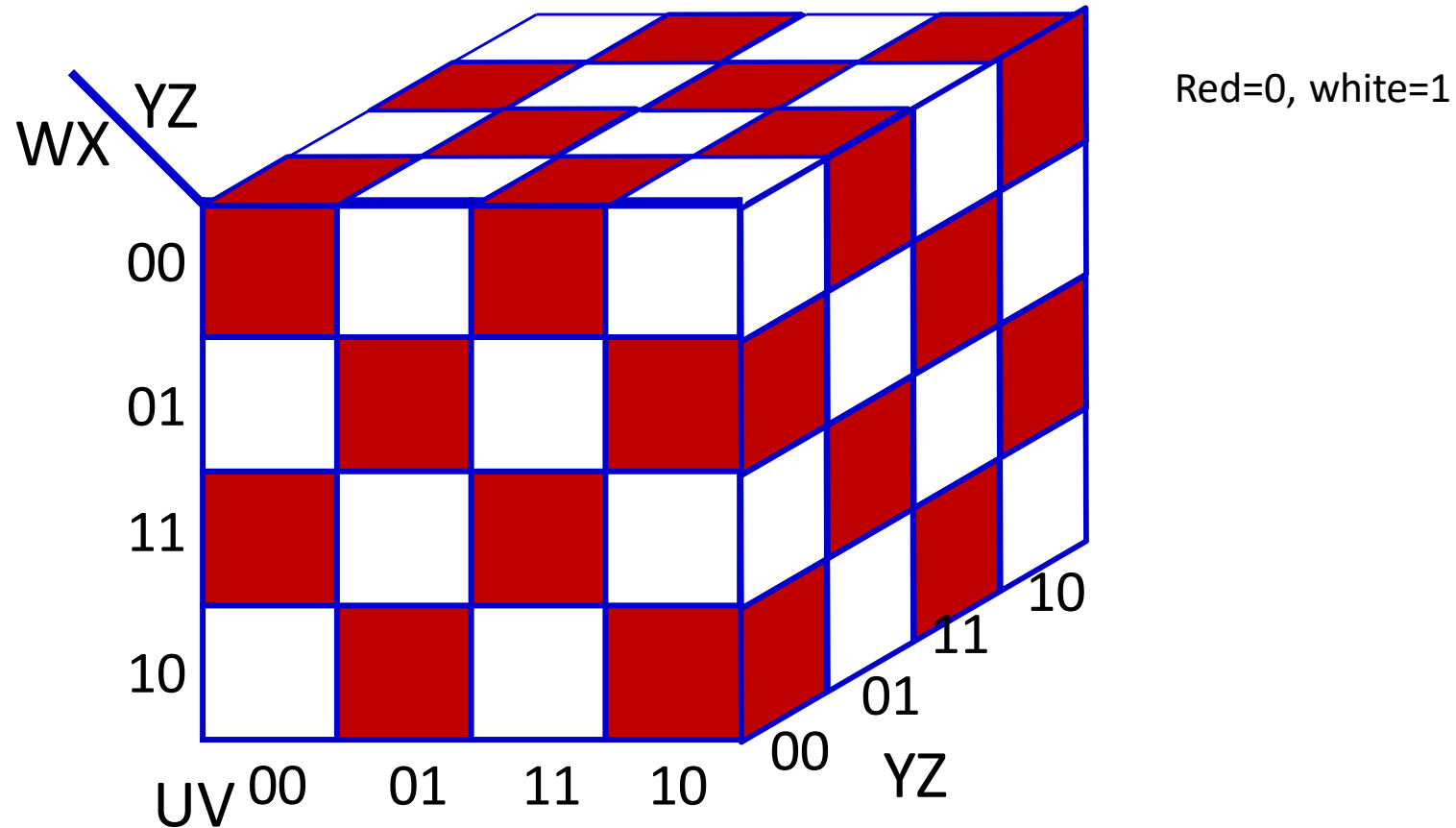
Largest irreducible DNF?

WX \ YZ	00	01	11	10
00	1	0	1	0
01	0	1	0	1
11	1	0	1	0
10	0	1	0	1

How many neurons in a DNF (one-hidden-layer) MLP for this Boolean function?

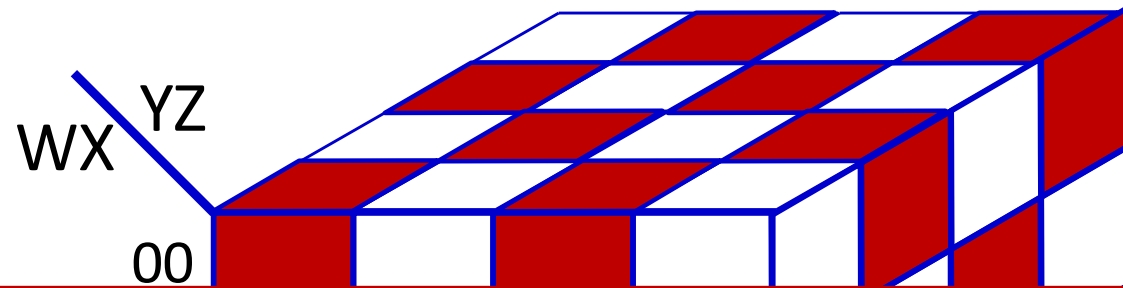
- What arrangement of ones and zeros simply cannot be reduced further?

Width of a one-hidden-layer Boolean MLP

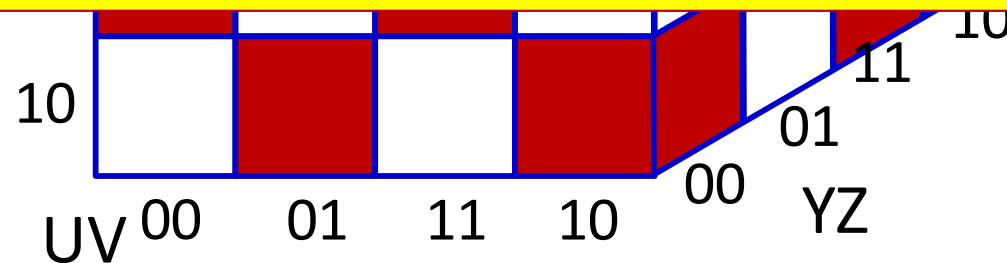


- How many neurons in a DNF (one-hidden-layer) MLP for this Boolean function of 6 variables?

Width of a one-hidden-layer Boolean MLP

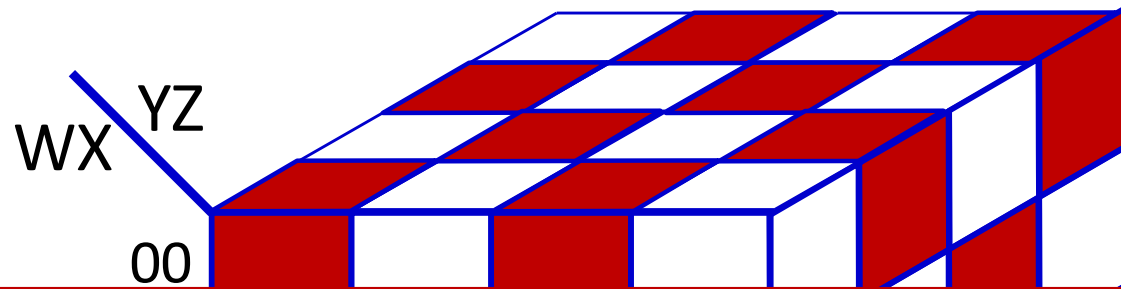


Can be generalized: Will require 2^{N-1} perceptrons in hidden layer
Exponential in N

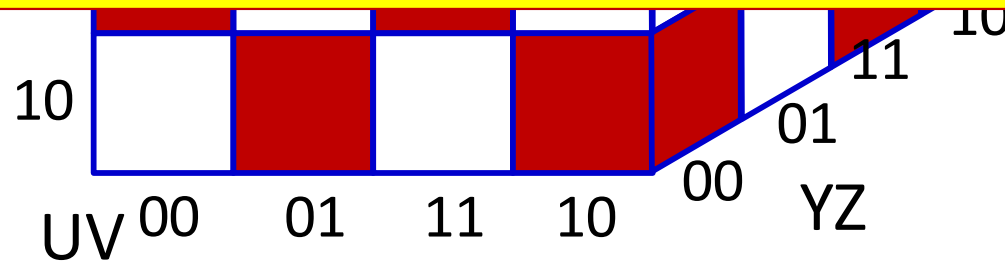


- How many neurons in a DNF (one-hidden-layer) MLP for this Boolean function

Width of a one-hidden-layer Boolean MLP



Can be generalized: Will require 2^{N-1} perceptrons in hidden layer
Exponential in N

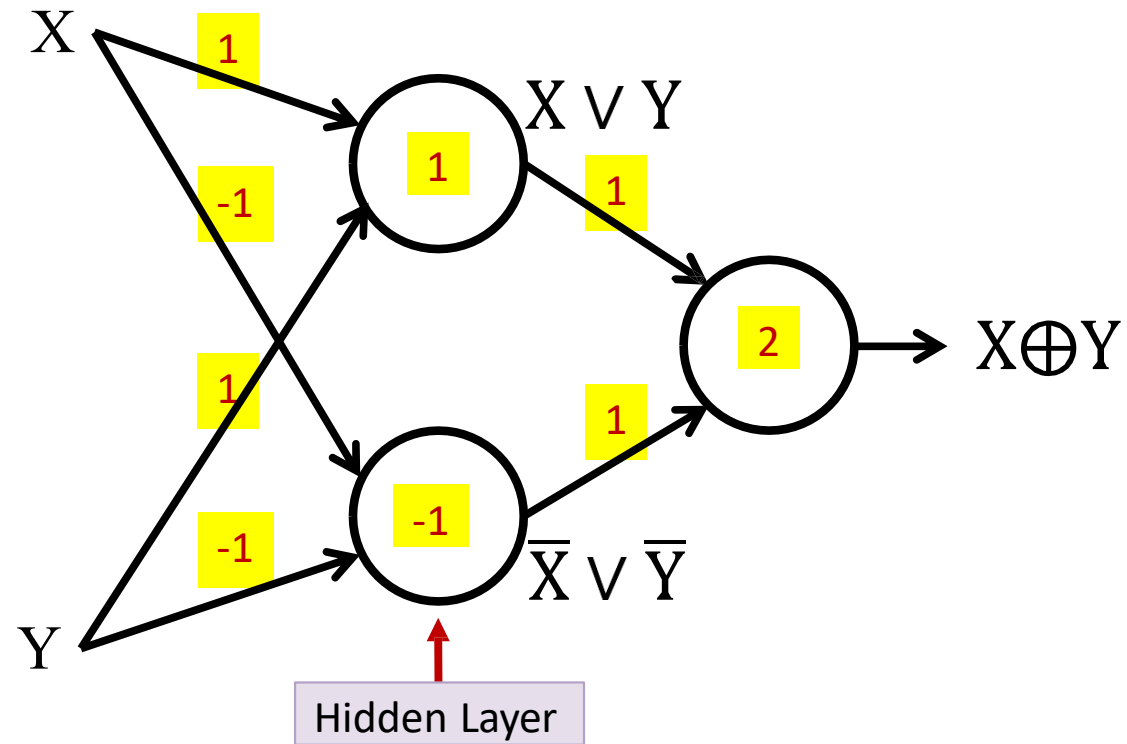


How many units if we use *multiple hidden layers*?

$$O = W \oplus X \oplus Y \oplus Z$$

$$O = U \oplus V \oplus W \oplus X \oplus Y \oplus Z$$

Multi-layer perceptron XOR

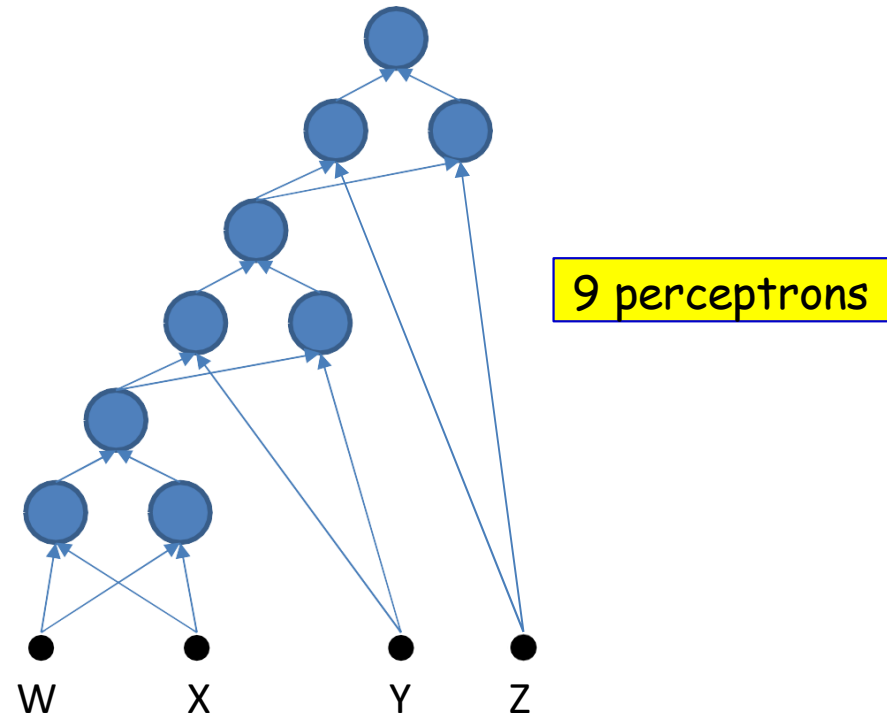


- An XOR takes three perceptrons

Size of a deep MLP

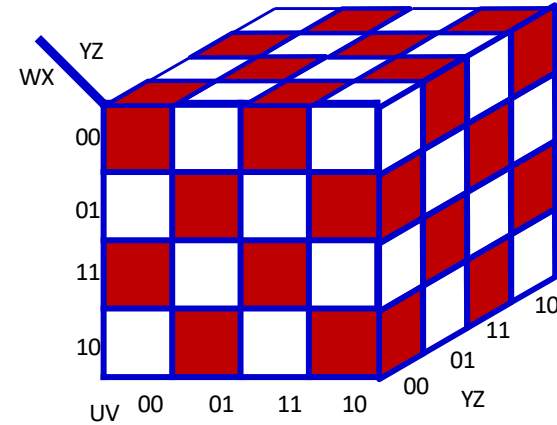
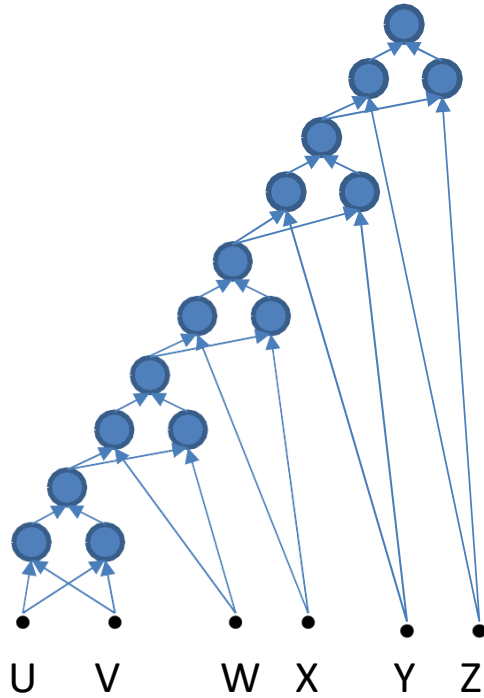
YZ \ WX	00	01	11	10
00	1	0	1	0
01	0	1	0	1
11	1	0	1	0
10	0	1	0	1

$$O = W \oplus X \oplus Y \oplus Z$$



- An XOR needs 3 perceptrons
- This network will require $3 \times 3 = 9$ perceptrons

Size of a deep MLP

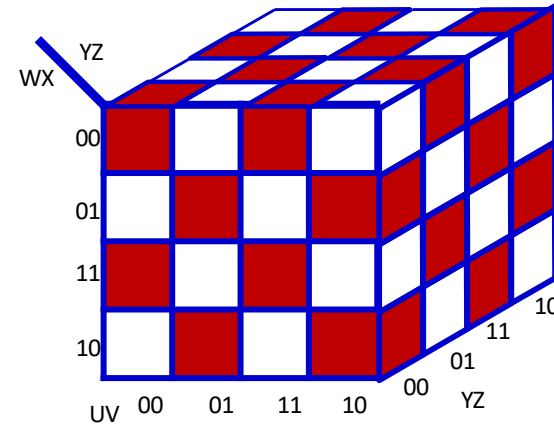
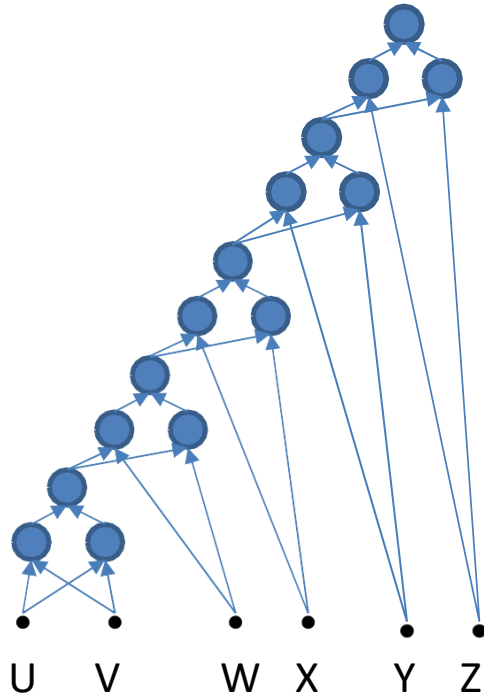


$$O = U \oplus V \oplus W \oplus X \oplus Y \oplus Z$$

15 perceptrons

- An XOR needs 3 perceptrons
- This network will require $3 \times 5 = 15$ perceptrons

Size of a deep MLP

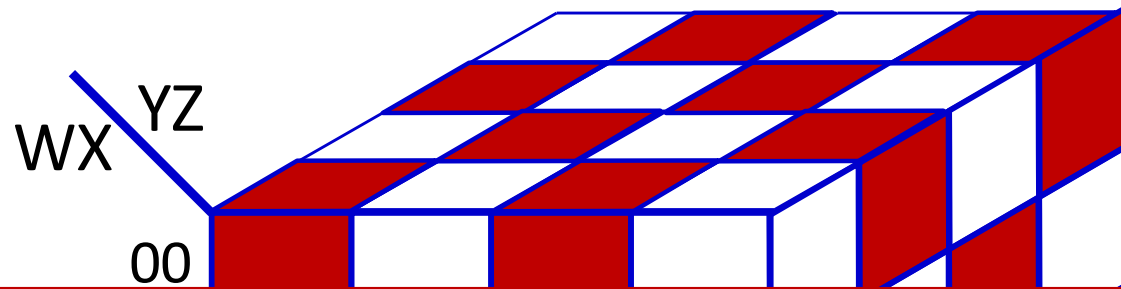


$$O = U \oplus V \oplus W \oplus X \oplus Y \oplus Z$$

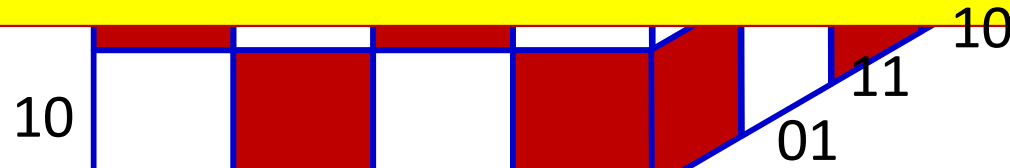
More generally, the XOR of N variables will require $3(N-1)$ perceptrons!!

- An XOR needs 3 perceptrons
- This network will require $3 \times 5 = 15$ perceptrons

One-hidden layer vs deep Boolean MLP



Single hidden layer: Will require $2^{N-1}+1$ perceptrons in all (including output unit)
Exponential in N

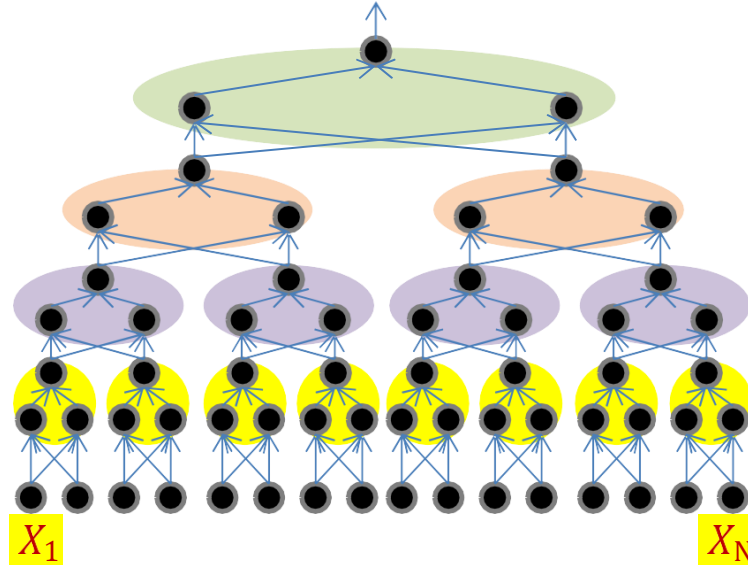


Will require $3(N-1)$ perceptrons in a deep network

Linear in N !!!

Can be arranged in only $2\log_2(N)$ layers

A better representation

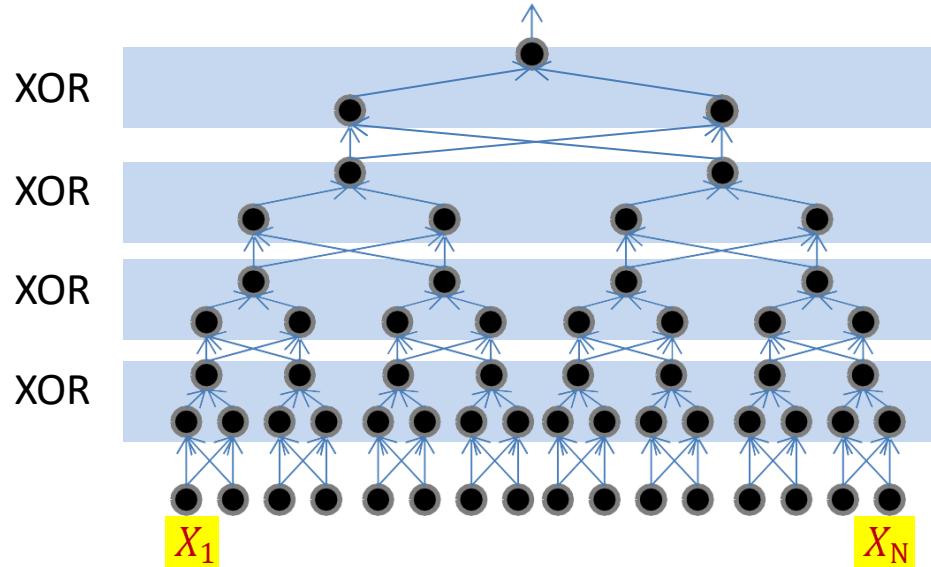


$$O = X_1 \oplus X_2 \oplus \cdots \oplus X_N$$

- Only $2 \log_2 N$ layers
 - By pairing terms
 - 2 layers per XOR

$$O = (((((X_1 \oplus X_2) \oplus (X_3 \oplus X_4)) \oplus ((X_5 \oplus X_6) \oplus (X_7 \oplus X_8)))) \oplus (((...$$

A better representation

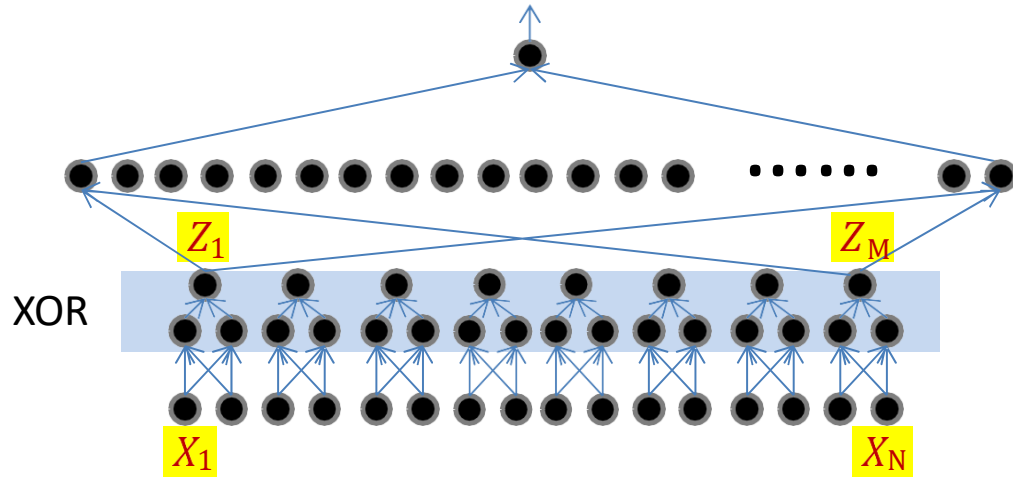


$$O = X_1 \oplus X_2 \oplus \cdots \oplus X_N$$

- Only $2 \log_2 N$ layers
 - By pairing terms
 - 2 layers per XOR

$$O = (((((X_1 \oplus X_2) \oplus (X_3 \oplus X_4)) \oplus ((X_5 \oplus X_6) \oplus (X_7 \oplus X_8))) \oplus (((\dots$$

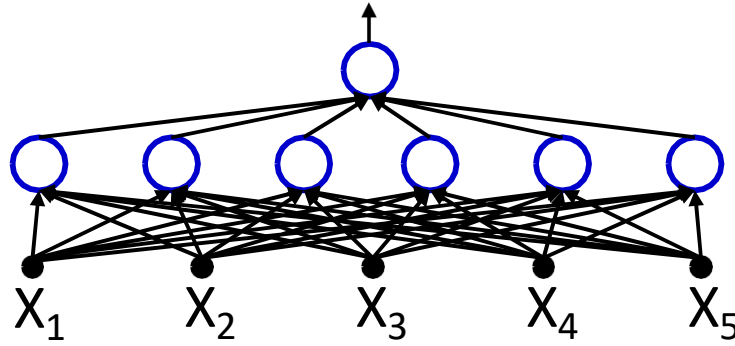
The challenge of depth



$$O = X_1 \oplus X_2 \oplus \dots \oplus X_N$$
$$= Z_1 \oplus Z_2 \oplus \dots \oplus Z_M$$

- Using only K hidden layers will require $O(2^{CN})$ neurons in the K th layer, where $C = 2^{-(K-1)/2}$
 - Because the output is the XOR of all the $N/2^{K-1/2}$ values output by the $K-1$ th hidden layer
 - **i.e. reducing the number of layers below the minimum will result in an exponentially sized network to express the function fully**
 - **A network with fewer than the minimum required number of neurons *cannot* model the function**

The actual number of parameters in a network



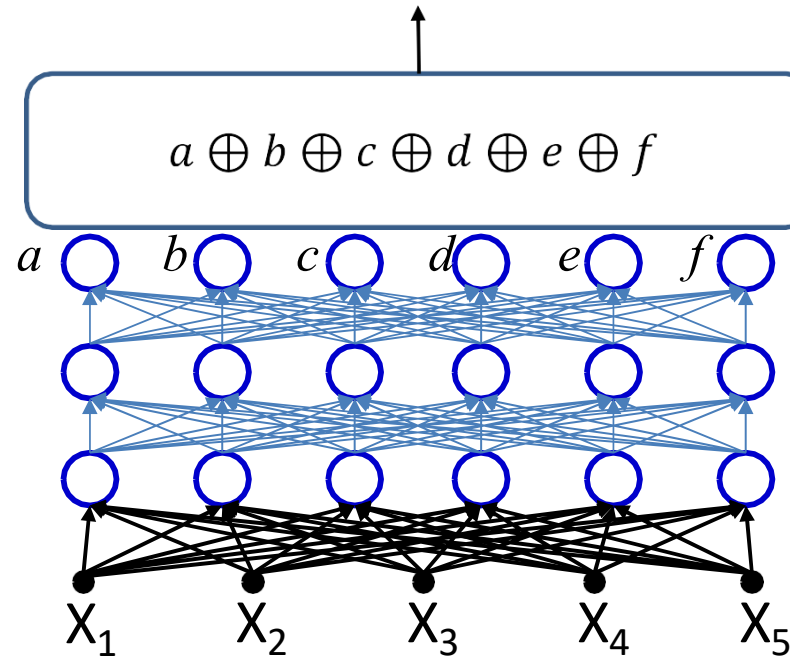
- The actual number of parameters in a network is the number of *connections*
 - In this example there are 30
- This is the number that really matters in software or hardware implementations
- Networks that require an exponential number of neurons will require an exponential number of weights..

Recap: The need for depth

- *Deep* Boolean MLPs that scale *linearly* with the number of inputs ...
- ... can become exponentially large if recast using only one hidden layer

The need for depth

The XORs could occur anywhere!



- An MLP for any function that can eventually be expressed as the XOR of a number of intermediate variables will require depth.
 - The XOR structure could occur in any layer
 - If you have a fixed depth from that point on, the network can grow exponentially in size.
- Having a few extra layers can greatly reduce network size

Depth vs Size in Boolean Circuits

- The XOR is really a parity problem
- Any *Boolean* parity circuit of depth d using AND, OR and NOT gates with unbounded fan-in must have size $2^{n^{1/d}}$
 - Parity, Circuits, and the Polynomial-Time Hierarchy, M. Furst, J. B. Saxe, and M. Sipser, Mathematical Systems Theory 1984
 - Alternately stated: $\text{parity} \notin AC^0$
 - Set of constant-depth polynomial size circuits of unbounded fan-in elements

Caveat : Not all Boolean functions..

- Not all Boolean circuits have such clear depth-vs-size tradeoff
- Shannon's theorem: For $n > 2$, there is a Boolean function of n variables that requires at least $2^n/n$ Boolean gates
 - More correctly, for large n , almost all n -input Boolean functions need more than $2^n/n$ Boolean gates
 - Regardless of depth
- Note: If all Boolean functions over n inputs could be computed using a circuit of size that is polynomial in n , $P = NP$!

Network size: summary

- An MLP is a universal Boolean function
- But can represent a given function only if
 - It is sufficiently wide
 - It is sufficiently deep
 - Depth can be traded off for (sometimes) exponential growth of the width of the network
- Optimal width and depth depend on the number of variables and the complexity of the Boolean function
 - Complexity: *minimal* number of terms in DNF formula to represent it

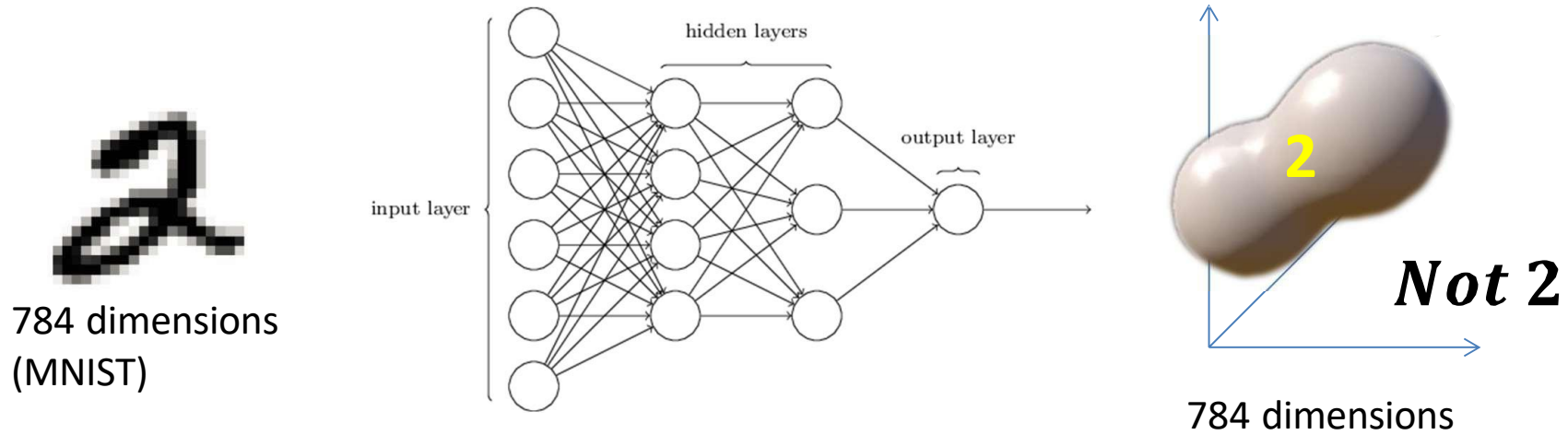
Story so far

- Multi-layer perceptrons are *Universal Boolean Machines*
- Even a network with a *single* hidden layer is a universal Boolean machine
 - But a single-layer network may require an exponentially large number of perceptrons
- Deeper networks may require far fewer neurons than shallower networks to express the same function
 - Could be *exponentially* smaller

Today

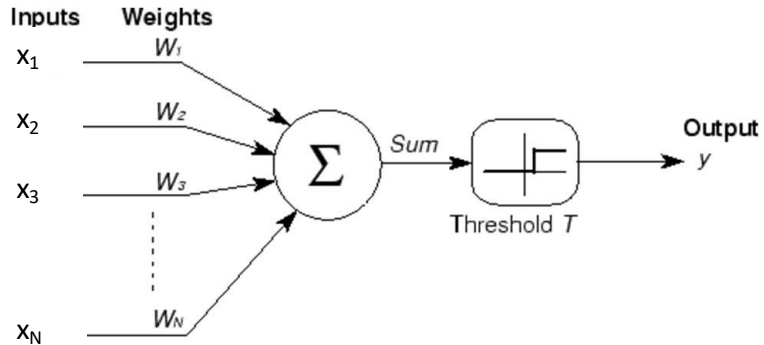
- Multi-layer Perceptrons as universal Boolean functions
 - The need for depth
- MLPs as universal classifiers
 - The need for depth
- MLPs as universal approximators
- A discussion of optimal depth and width

Recap: The MLP as a classifier



- MLP as a function over real inputs
- MLP as a function that finds a complex “decision boundary” over a space of *reals*

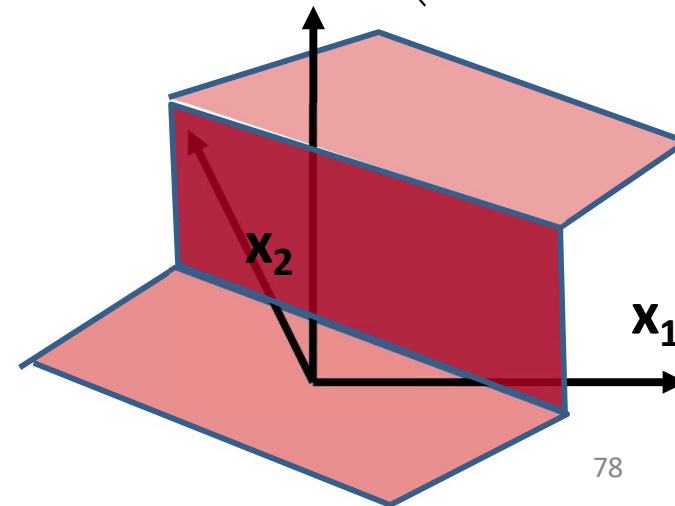
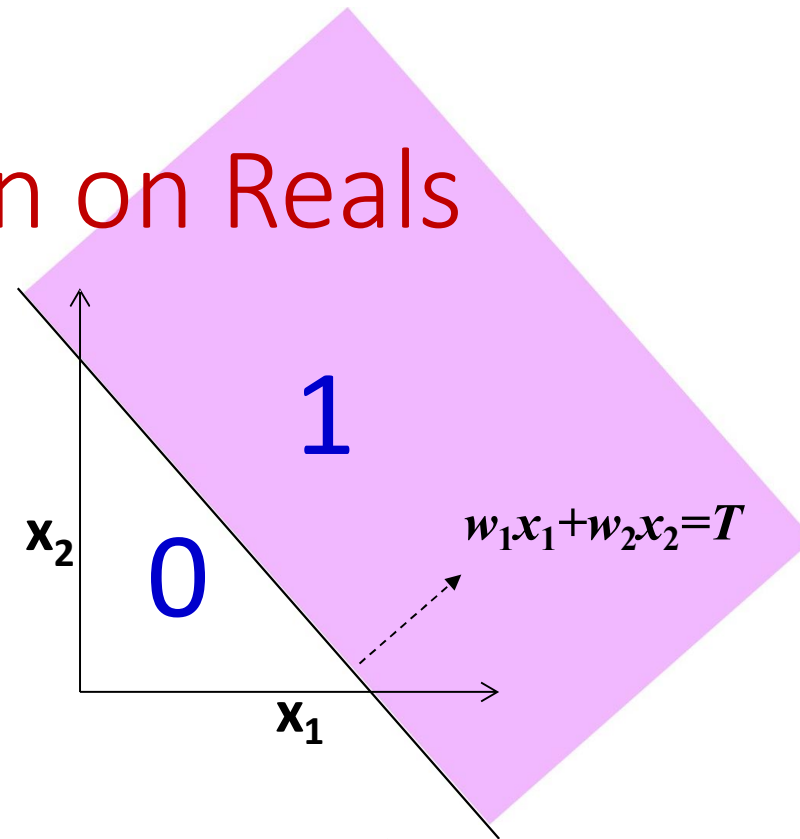
A Perceptron on Reals



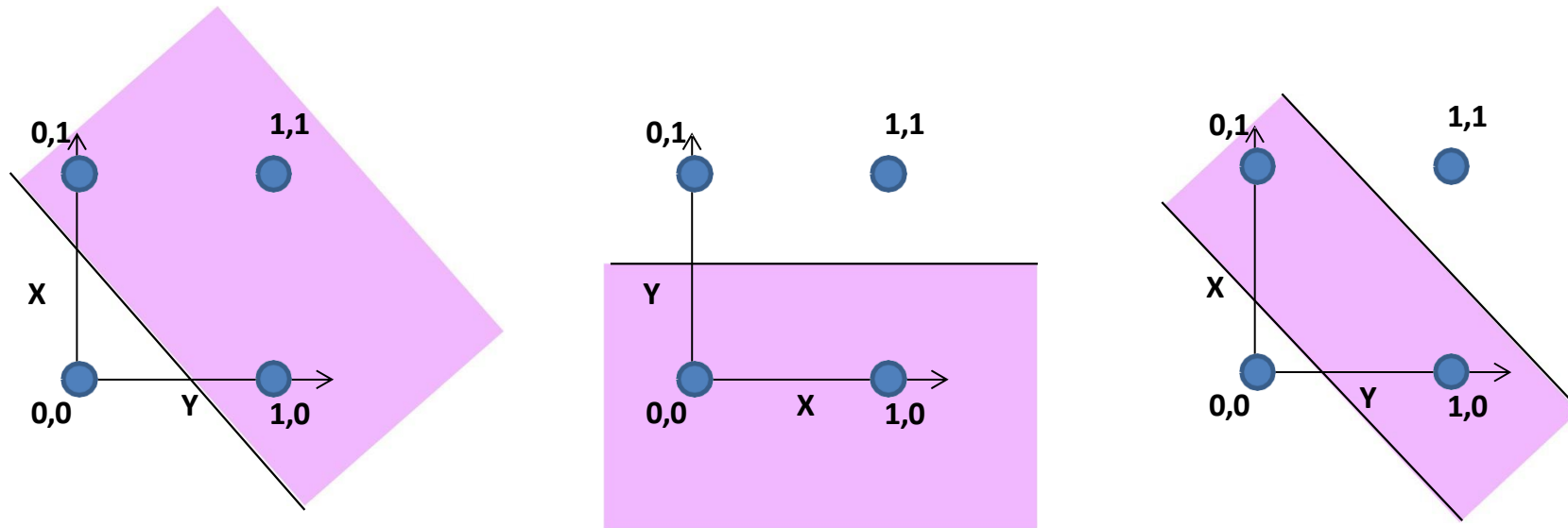
$$y = \begin{cases} 1 & \text{if } \sum_i w_i x_i \geq T \\ 0 & \text{else} \end{cases}$$

- A perceptron operates on *real*-valued vectors

– This is a *linear classifier*

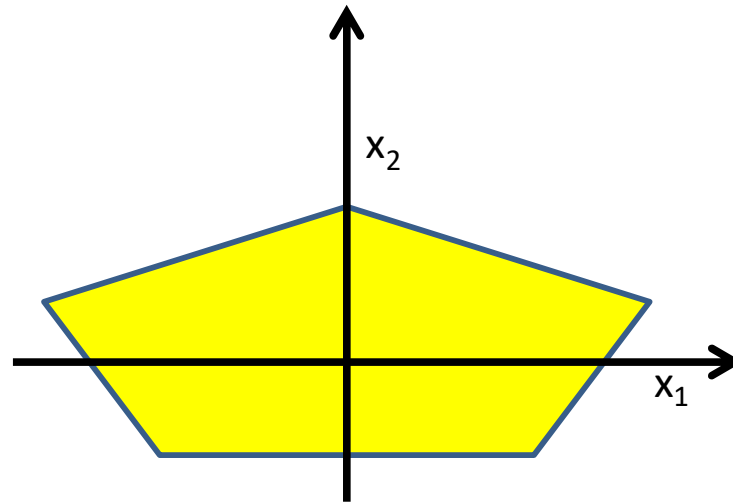


Boolean functions with a real perceptron



- Boolean perceptrons are also linear classifiers
 - Purple regions are 1

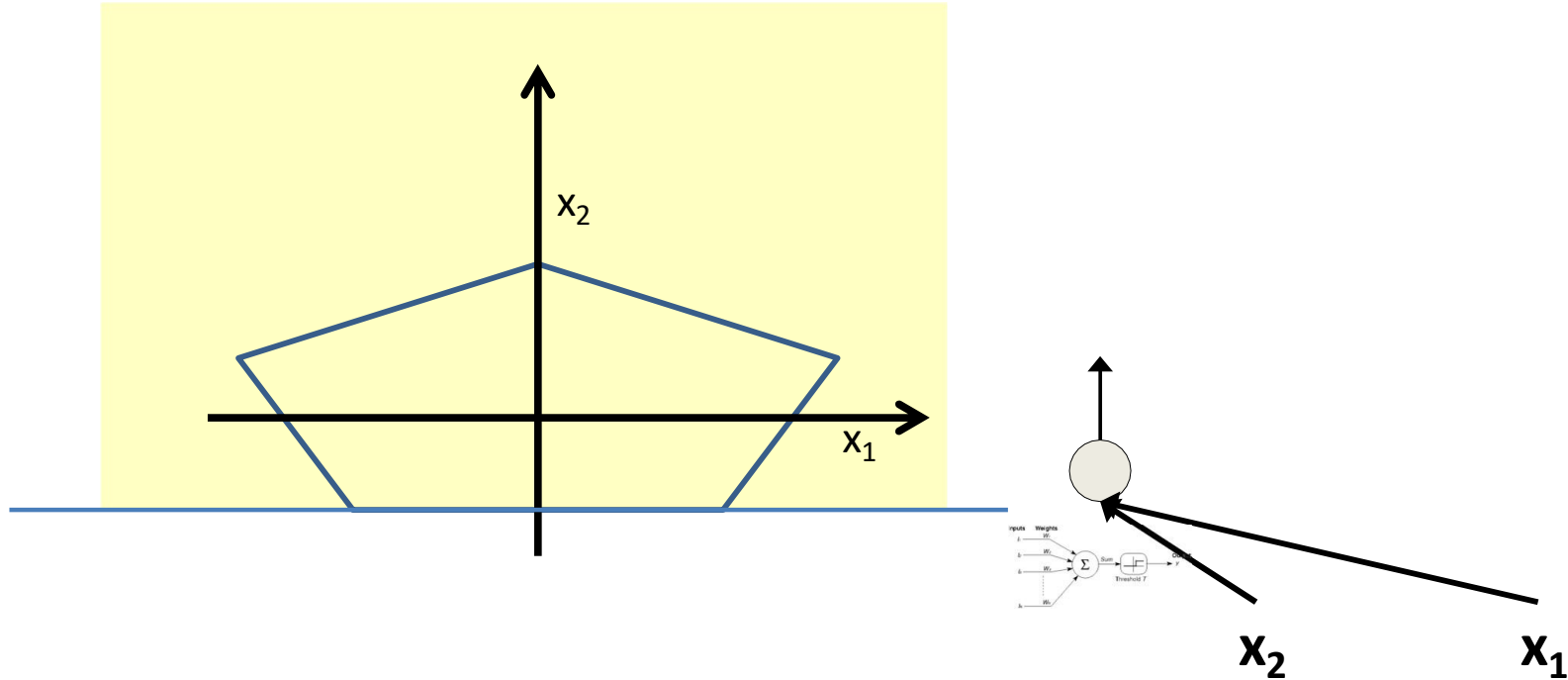
Composing complicated “decision” boundaries



Can now be composed into “networks” to compute arbitrary classification “boundaries”

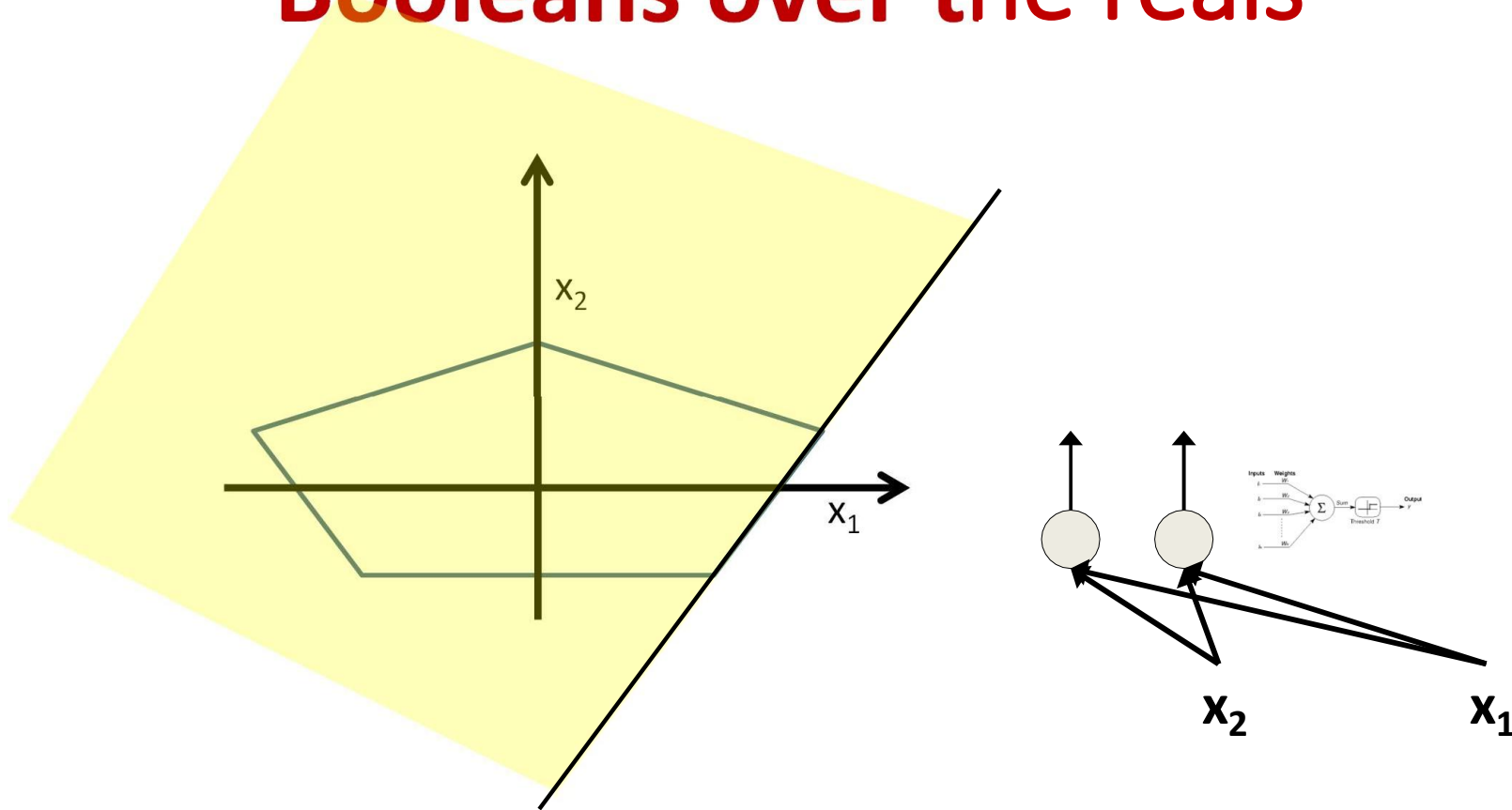
- Build a network of units with a single output that fires if the input is in the coloured area

Booleans over the reals



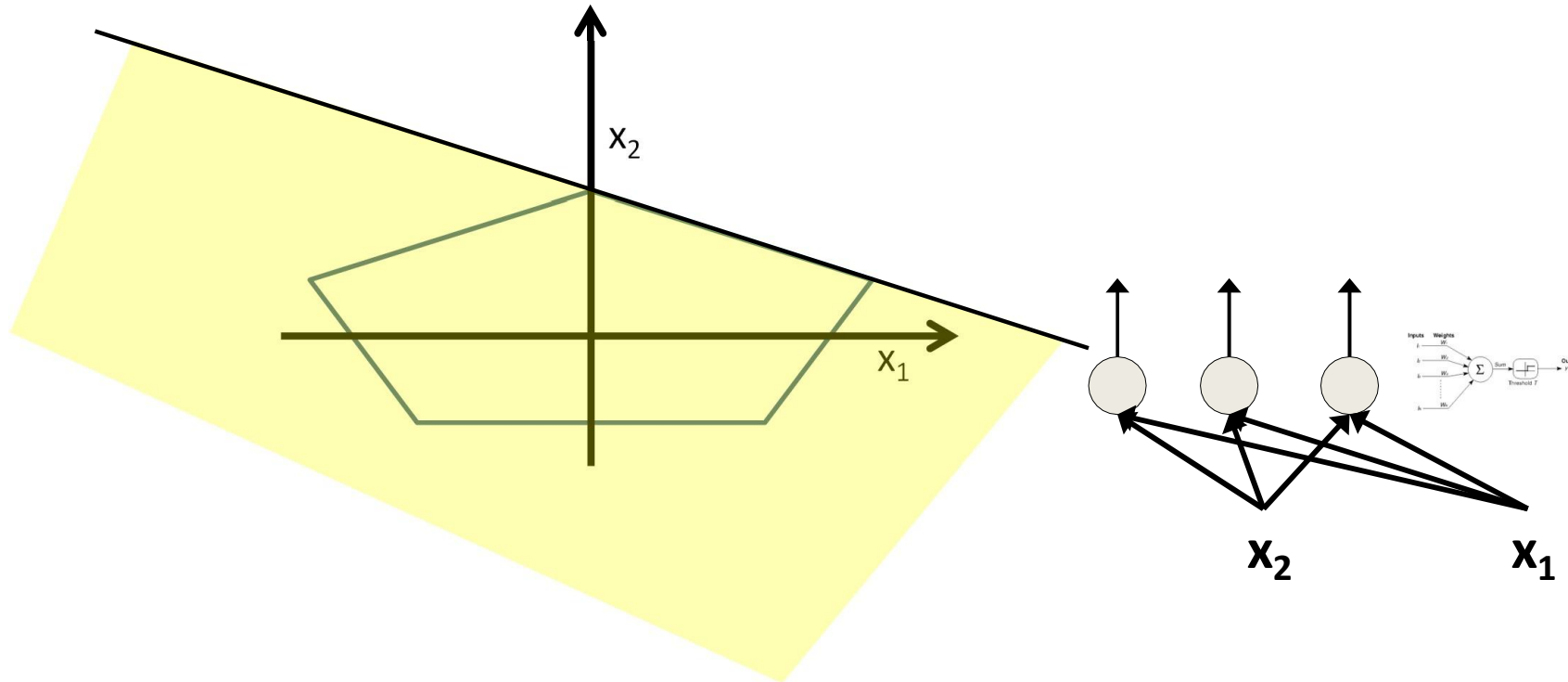
- The network must fire if the input is in the coloured area

Booleans over the reals



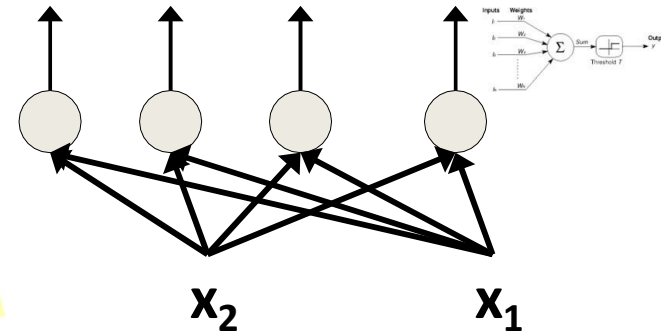
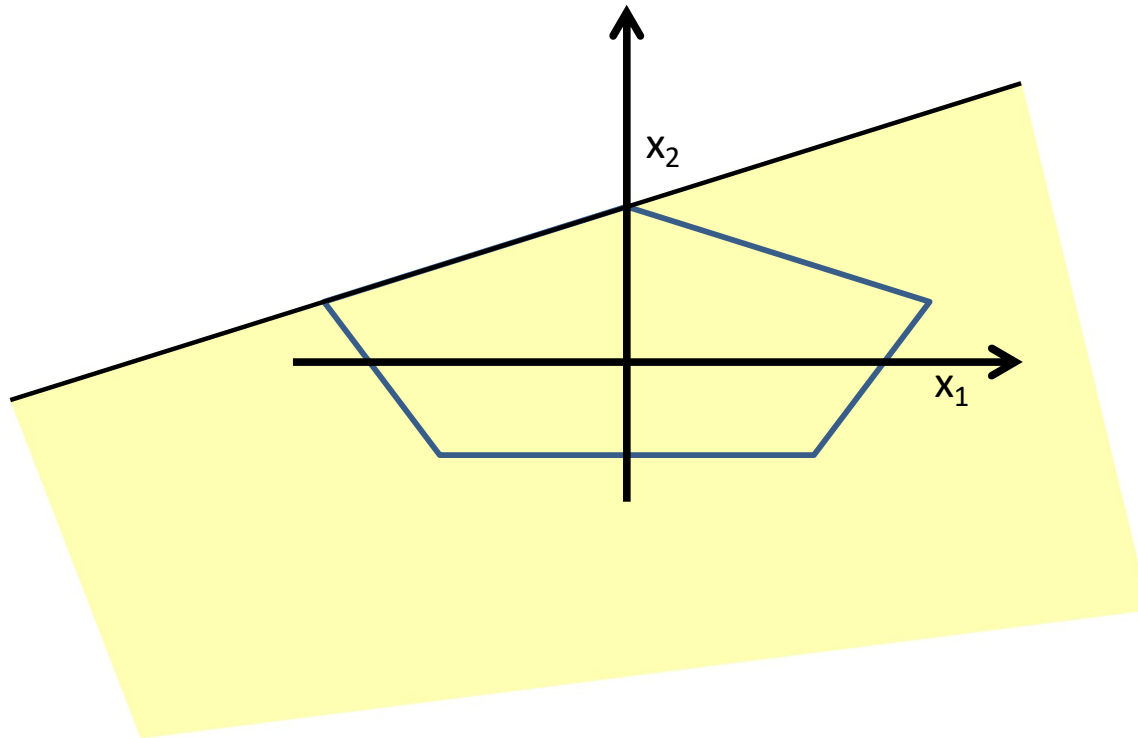
- The network must fire if the input is in the coloured area

Booleans over the reals



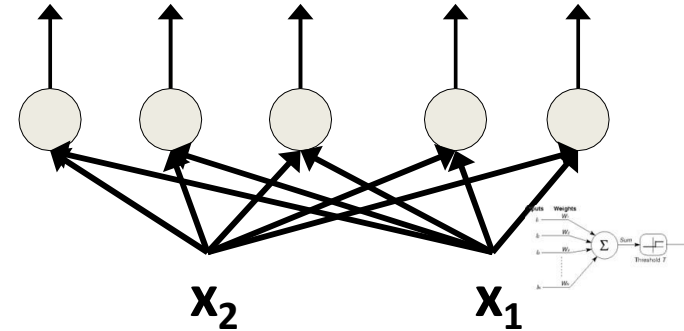
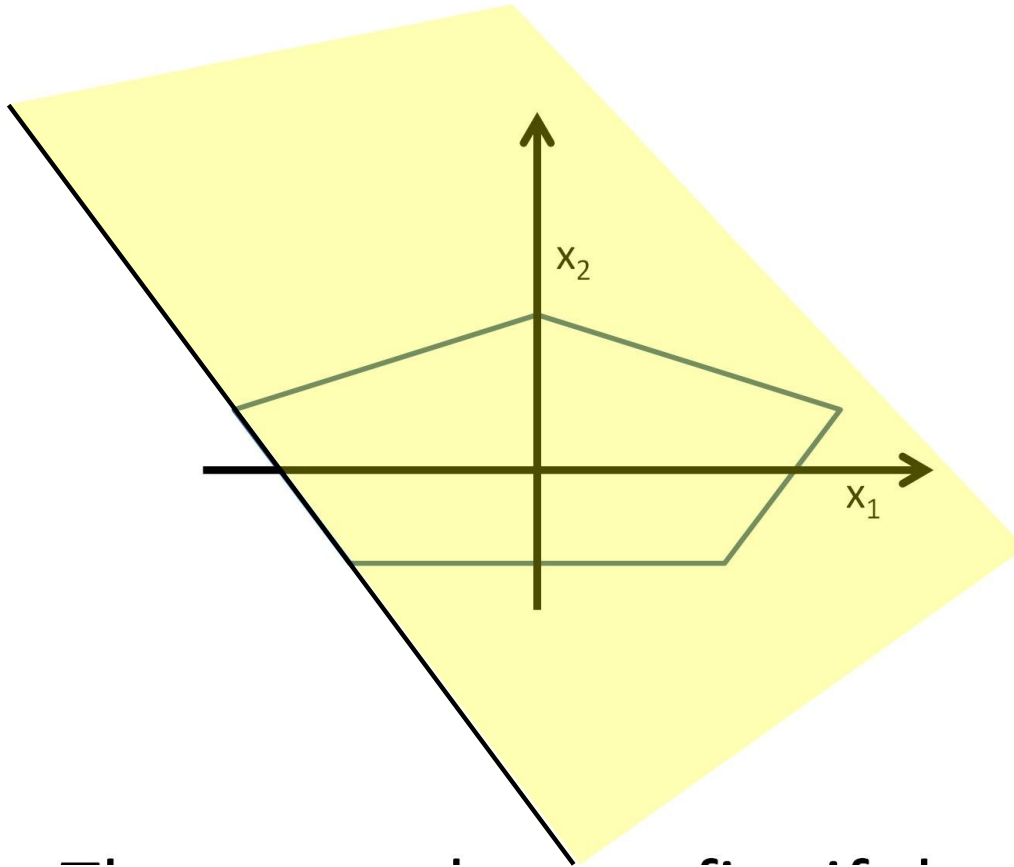
- The network must fire if the input is in the coloured area

Booleans over the reals



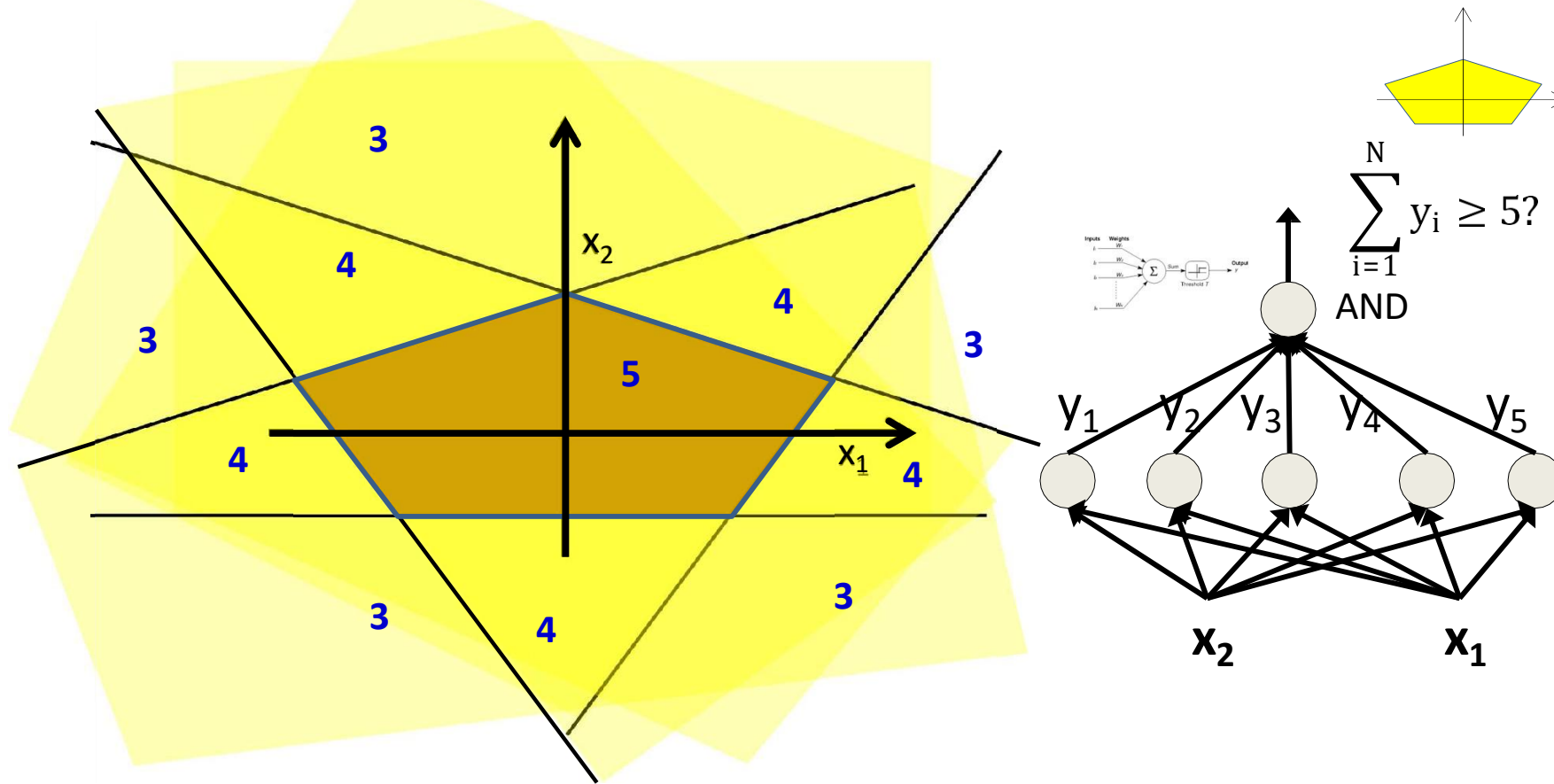
- The network must fire if the input is in the coloured area

Booleans over the reals



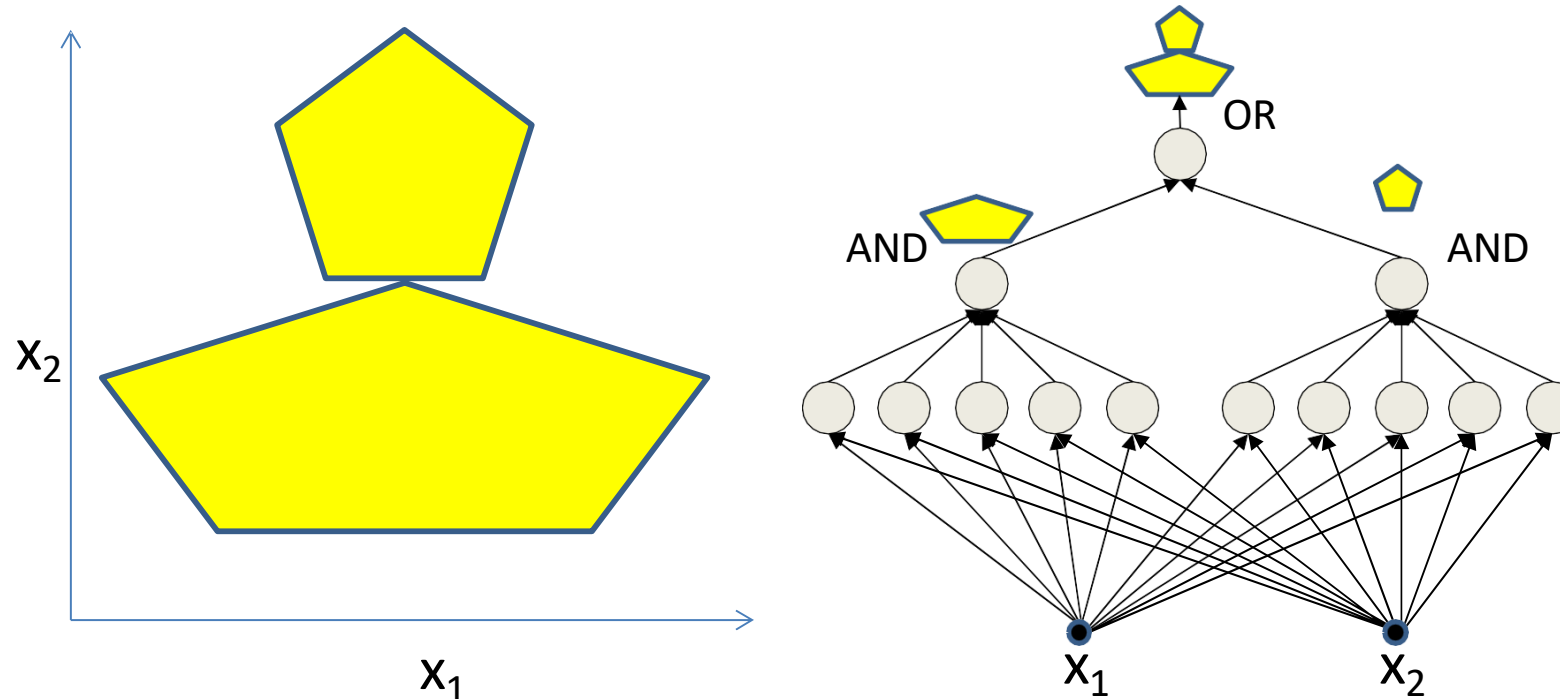
- The network must fire if the input is in the coloured area

Booleans over the reals



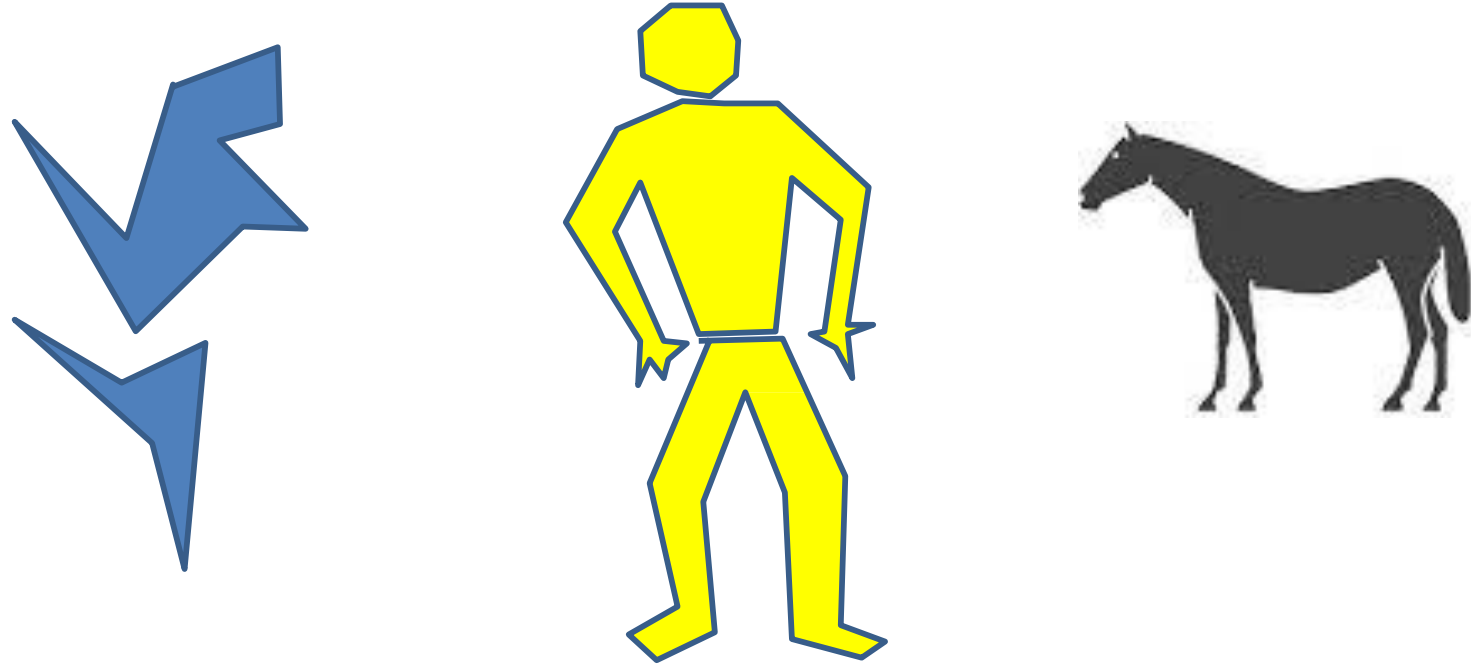
- The network must fire if the input is in the coloured area
 - The AND compares the sum of the hidden outputs to 5

More complex decision boundaries



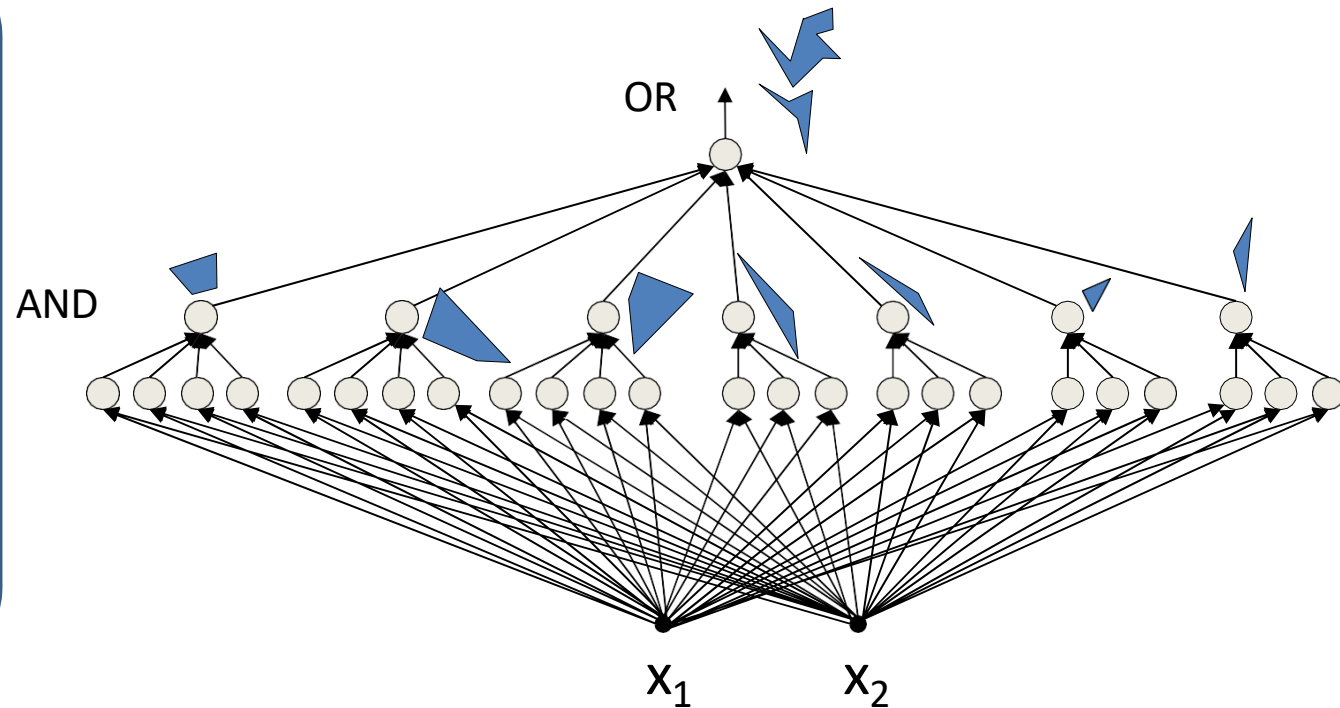
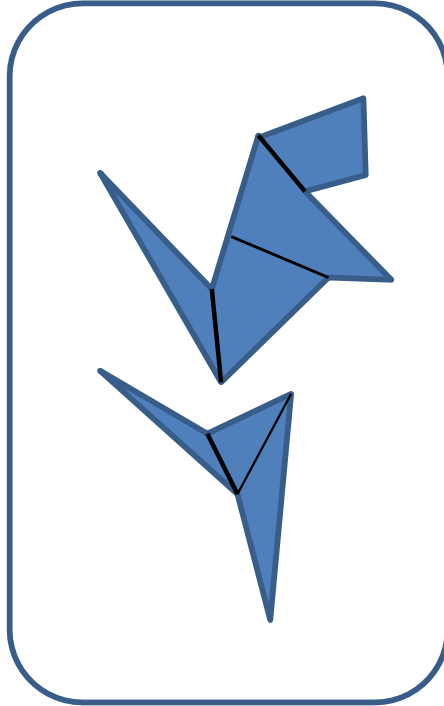
- Network to fire if the input is in the yellow area
 - “OR” two polygons
 - A third layer is required

Complex decision boundaries



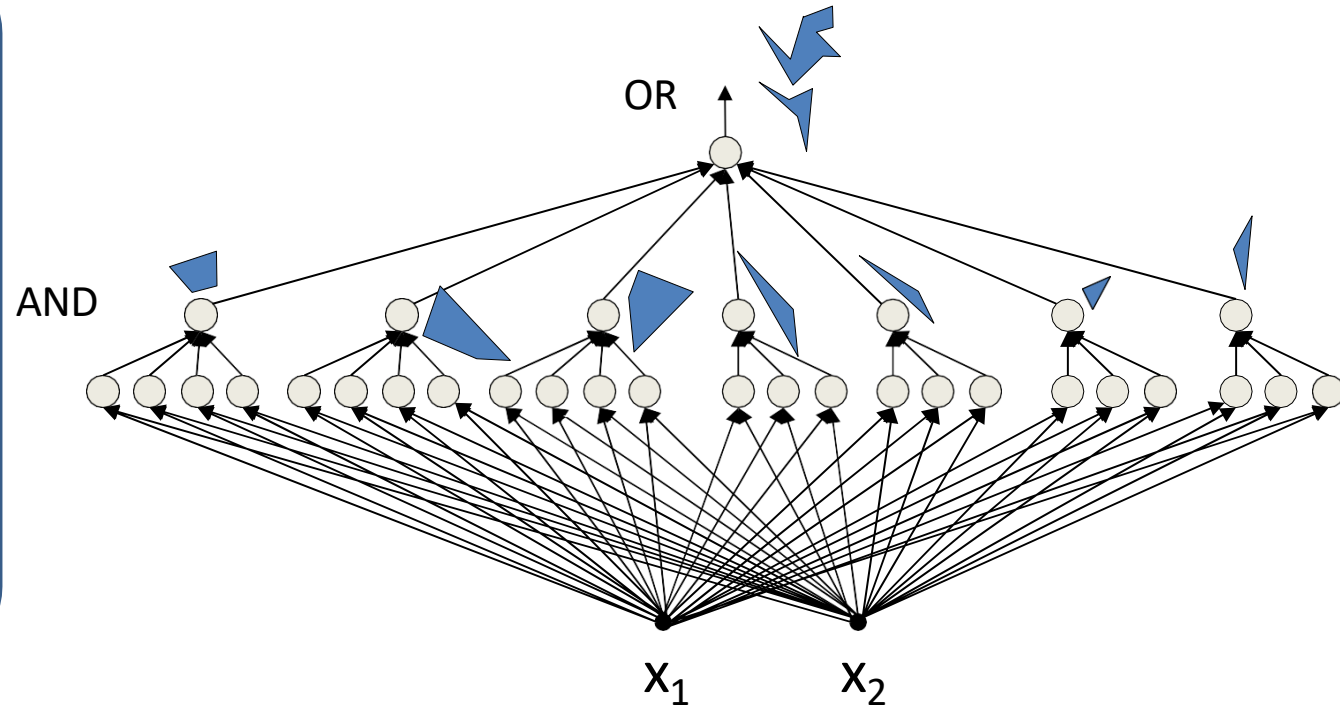
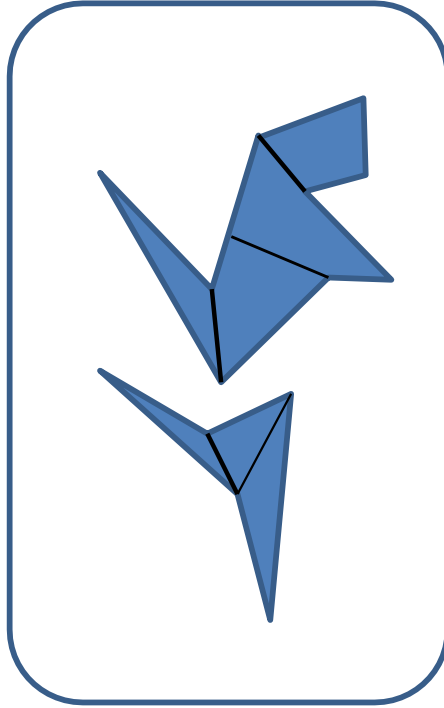
- Can compose *arbitrarily* complex decision boundaries

Complex decision boundaries



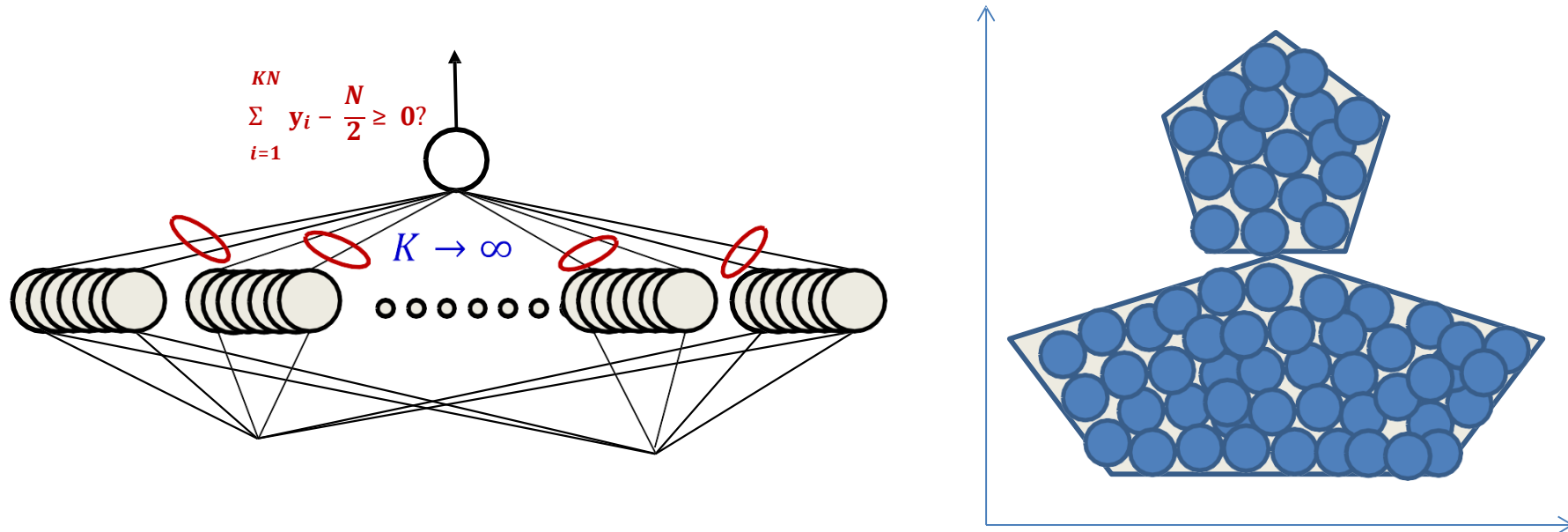
- Can compose *arbitrarily* complex decision boundaries

Complex decision boundaries



- Can compose *arbitrarily* complex decision boundaries
 - With *only one hidden layer*!

MLP: Universal classifier

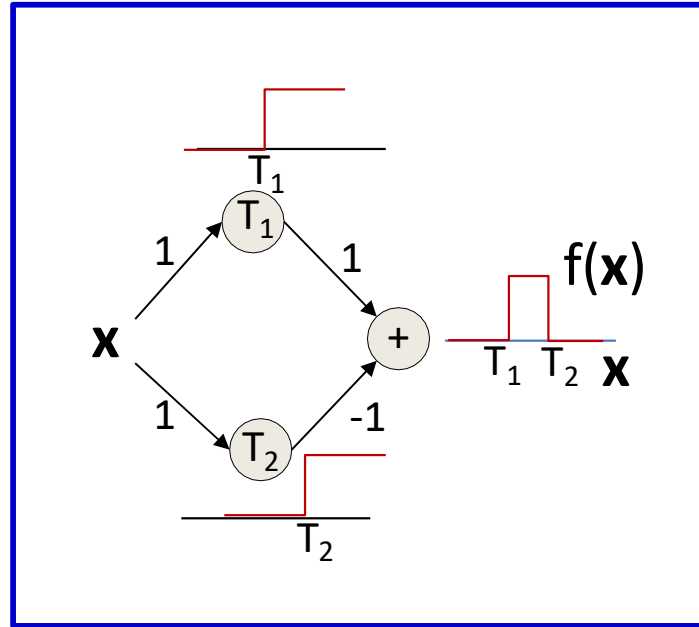


- MLPs can capture *any* classification boundary
- A *one-hidden-layer MLP* can model any classification boundary
- *MLPs are universal classifiers*

Today

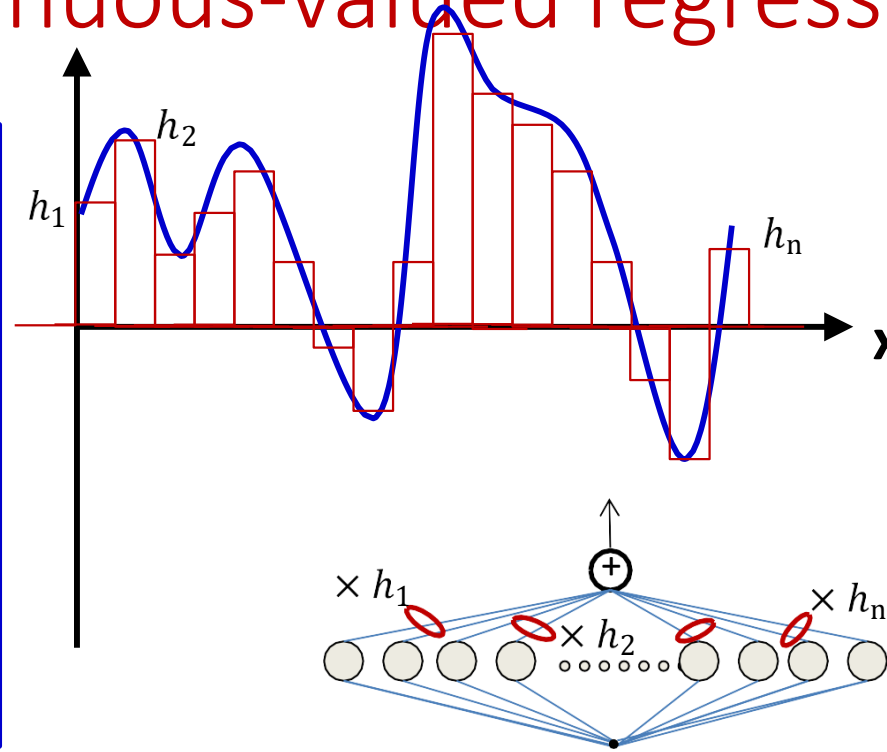
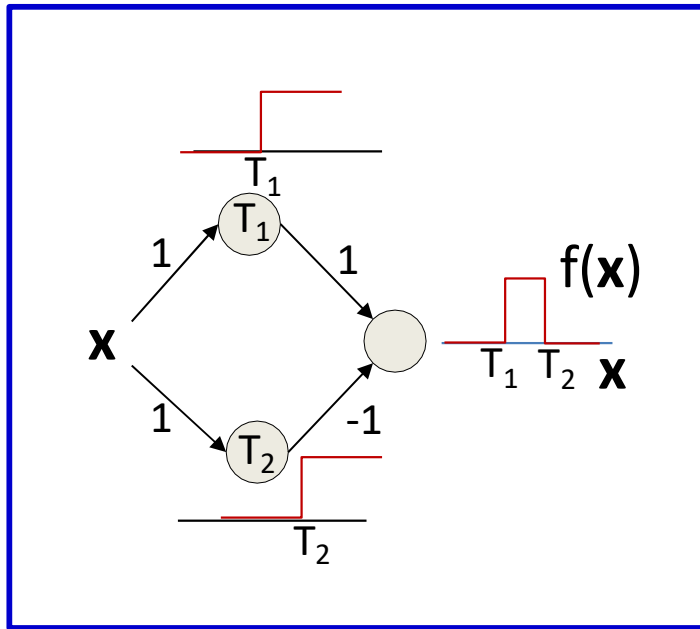
- Multi-layer Perceptrons as universal Boolean functions
 - The need for depth
- MLPs as universal classifiers
 - The need for depth
- MLPs as universal approximators

MLP as a continuous-valued regression



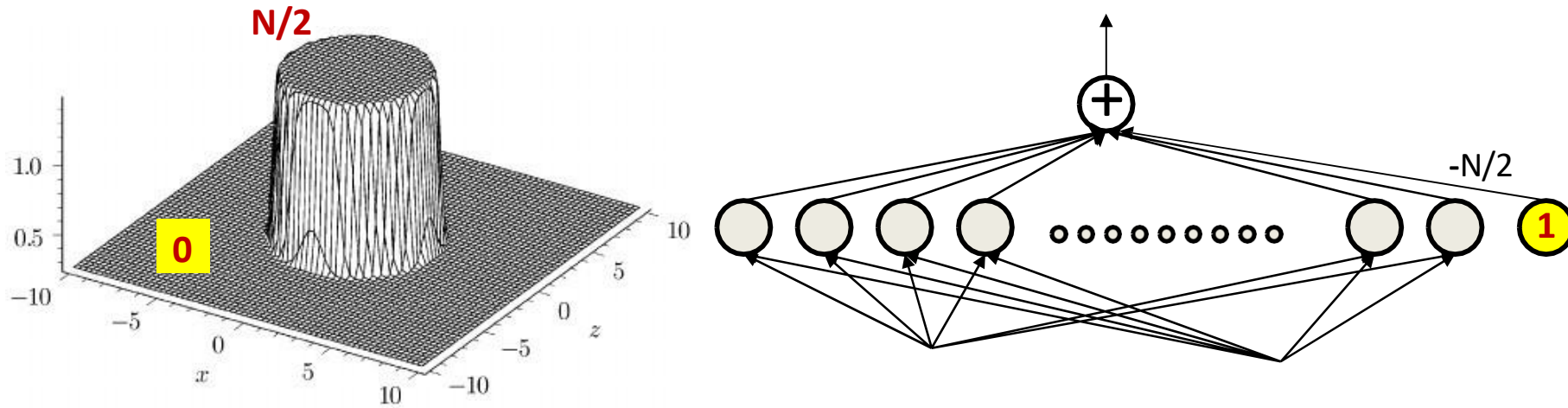
- A simple 3-unit MLP with a “summing” output unit can generate a “square pulse” over an input
 - Output is 1 only if the input lies between T_1 and T_2
 - T_1 and T_2 can be arbitrarily specified

MLP as a continuous-valued regression



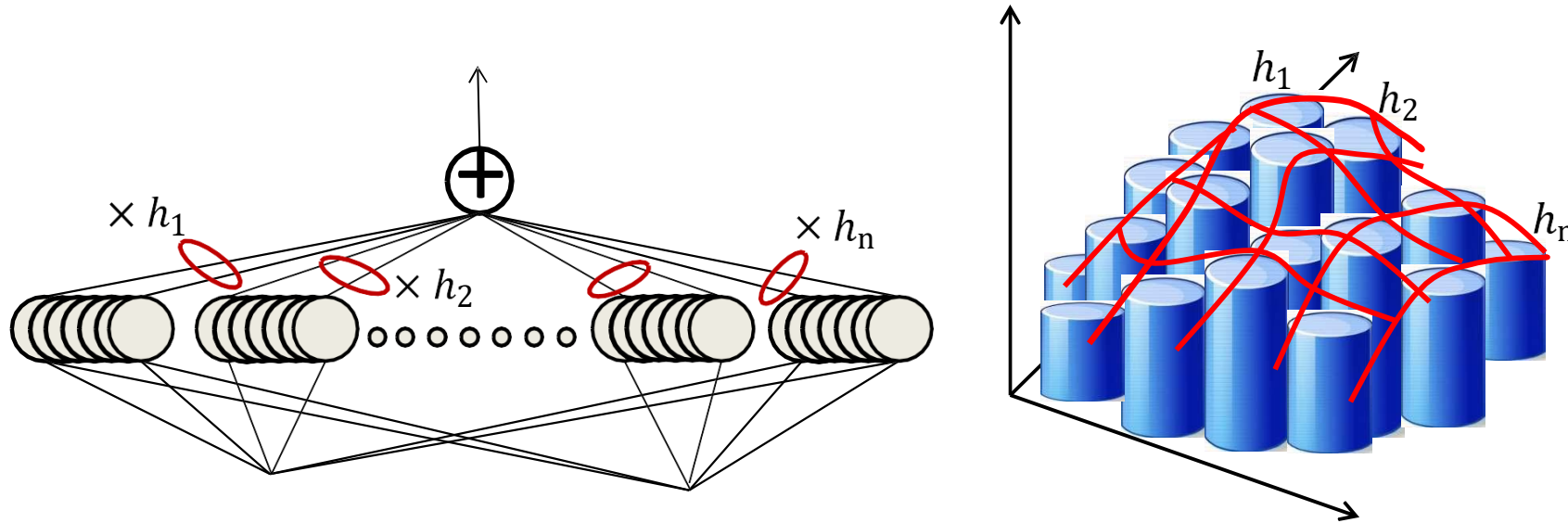
- A simple 3-unit MLP can generate a “square pulse” over an input
- **An MLP with many units can model an arbitrary function over an input**
 - To arbitrary precision
 - Simply make the individual pulses narrower
- ***A one-hidden-layer MLP can model an arbitrary function of a single input***

For higher dimensions



- An MLP can compose a cylinder
 - $N/2$ in the circle, 0 outside

MLP as a continuous-valued function



- MLPs can actually compose arbitrary functions in any number of dimensions!
 - Even with only one hidden layer
 - As sums of scaled and shifted cylinders
 - To arbitrary precision
 - By making the cylinders thinner
 - **The MLP is a universal approximator!**

Lessons today

- MLPs are universal Boolean function
- MLPs are universal classifiers
- MLPs are universal function approximators
- *A single-layer* MLP can approximate anything to arbitrary precision
 - But could be exponentially or even infinitely wide in its inputs size
- Deeper MLPs can achieve the same precision with far fewer neurons
 - Deeper networks are more expressive
 - More graded activation functions result in more expressive networks